ASSIGNMENT 1

CO1: *Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation*

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks - 50)

Nature of Assignment: Individual
Duration: 1 Week
Submission Date: **12/11/2026**

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA09-<RegisterNumber>-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator). • Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the Java source code, input files and output screenshots.
- Include a README.md with your name, register number, Java version, the steps to compile and run each program, and a short note of the result of each task. • Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted. • Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: **192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 1 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Campus Smart-Parking System

The university campus has a multi-level parking facility used by students, staff and visitors. The facility has several floors, and each floor has a fixed number of numbered slots. Vehicles entering the campus may be two-wheelers, cars or electric vehicles (EVs). EVs may also use charging slots, which are charged separately.

When a vehicle enters, the system must allocate a suitable free slot, issue a parking ticket with a unique ticket number, and record the entry time. When the vehicle leaves, the system must release the slot and compute the parking fee. Fees depend on the vehicle type and the duration
of parking. Staff and students holding a valid monthly pass park free of charge, while visitors pay the full fee.

The facility may contain:

1 floor × 20 slots → 3 floors × 50 slots → 5 floors × 100 slots

You are required to analyse the problem, design an object-oriented model for it, implement the model in Java and validate it. Your design must remain correct and easy to extend as the facility and its rules grow.

Design Challenge:

The university now wants to add new vehicle categories and a new pricing rule without rewriting the classes that already work.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects4 Marks

Bloom's Level: K4 – Analyse

Analyse the parking problem and identify the objects that a Java program must model.

Discuss:

- The inputs the system receives and the outputs it must produce • The nouns in the problem that become candidate classes, and the verbs that become methods
- The responsibilities of each candidate class
- The business rules every valid operation must satisfy (for example: a slot cannot hold two vehicles; a ticket number is never reused)
- Any assumptions you make about the facility

You are expected to **justify why each candidate class is needed**, not merely list names.

Alternative Concept and Complexity Analysis

In your design each vehicle is an object. If the vehicles were instead stored as plain Strings in parallel arrays (regNo[], type[], entryTime[]), what would the solution look like? Compare both designs for finding a vehicle by registration number and for adding a new vehicle attribute. Analyse the time complexity of the search and the space used per vehicle in each design.

Evidence Expected

A clear problem analysis with candidate classes, their responsibilities, business rules and stated assumptions.

TASK 2 – Choose the Data Representation 3 Marks**Bloom's Level:** K4 –

Analyse

Decide how the data of the system will be represented using Java data types and arrays.
Discuss:

- The data type chosen for each field (slot number, vehicle number, entry time, duration, fee) and why
- How the floors and slots will be represented (one-dimensional, two-dimensional or jagged array) and why
- Where type conversion or casting is required (for example: converting minutes to hours for billing) and the risk of losing precision
- How you will read input (Scanner or command-line arguments)

Show at least one example where **integer division or an incorrect cast would produce a wrong fee**, and how your design avoids it.

Alternative Concept and Complexity Analysis

Instead of a jagged array String[floor][slot], suppose the slots are stored in a single one dimensional array using index = floor × slotsPerFloor + slot, or in an ArrayList<String[]>. What would change in your solution when floors have different numbers of slots? Analyse the time to locate a slot and the space required in each representation.

Evidence Expected

Justified choice of data types and array structure, with an identified casting risk and its resolution.

TASK 3 – Design the Control Logic 4 Marks

Bloom's Level: K4 – Analyse

Design the decision and looping logic for the two core operations: **allocating a free slot** and **computing the parking fee**.

Discuss:

- How the program searches the floors and slots for a suitable free slot, and when the search stops
- How the fee slabs (for example: first hour, each additional hour, daily maximum) are expressed using if–else or switch
- How break, continue and return are used to avoid unnecessary work • What happens when no suitable slot is free

Write **pseudocode for both operations** following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing Java code.

Complete the Code

Complete the class below and execute it. Do not change the main() method.

```
public class ParkingFacility {
 // slots[floor][slot] holds a vehicle number, or null if the slot is free
private final String[][] slots;

 public ParkingFacility(int[] slotsPerFloor) {
 slots = new String[slotsPerFloor.length][];
 for (int f = 0; f < slotsPerFloor.length; f++) {  slots[f] = new
String[slotsPerFloor[f]];  }
 }

 /** Returns {floor, slot} of the first free slot, or null if it is full. */
public int[] allocate(String vehicleNo) {
 // TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first //
free slot and return {floor, slot}

 return null;
 }

 /** Frees a slot. Returns false if the position is invalid or already free. */
public boolean release(int floor, int slot) {
 // TODO 2: Validate floor and slot, free the slot and return true; otherwise //
return false

 }

 public static void main(String[] args) {
 // floor 0 has 2 slots, floor 1 has 1 slot
 ParkingFacility p = new ParkingFacility(new int[]{2, 1});
System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "  + p.release(5, 0));
 System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444"))); }
}
```

Expected output:

```
 [0, 0]
 [0, 1]
 [1, 0]
null
true false false
 [0, 1]
```

Alternative Concept and Complexity Analysis

Your allocation scans every floor and slot until a free one is found. If, instead, you maintained a count of free slots per floor and a stack of free slot numbers, what would the allocation and release logic become? Analyse the time complexity of allocate() and release() and the extra space needed in both approaches for F floors and S slots per floor.

Evidence Expected

Complete pseudocode for slot allocation and fee computation with correct decision and loop logic.

TASK 4 – Design Classes, Objects and Constructors4 Marks**Bloom's Level:** K4 – Analyse

Design the classes identified in Task 1 and decide how their objects will be created.
Discuss:

- The fields and methods of each class
- Default, parameterised and overloaded constructors, and when each is used • Use of the this keyword and constructor chaining to avoid repeated code • Whether a copy constructor is needed anywhere (for example: re-issuing a duplicate ticket) and why
- How objects are stored in memory (reference variables, stack and heap)

Alternative Concept and Complexity Analysis

You used overloaded constructors and constructor chaining. If objects were instead created with a default constructor followed by setter calls, or through static factory methods, what would the solution be? Compare how each approach guarantees a valid object, and analyse the time and space cost of creating n ParkingTicket objects in each approach.

Evidence Expected

Class definitions with justified constructors, use of this and constructor chaining, and an explanation of object creation.

TASK 5 – Apply static Members and the Object Life Cycle3 Marks

Bloom's Level: K4 – Analyse

Identify the data and behaviour that belong to the class as a whole rather than to individual objects.

Discuss:

- Where static variables are needed (for example: a ticket counter, total occupancy) and why an instance variable would be wrong
- Where static methods or a static block are appropriate
- What happens to a ParkingTicket object after the vehicle leaves, and when it becomes eligible for garbage collection
- How objects are passed to and returned from methods in your design

Alternative Concept and Complexity Analysis

The ticket counter is a static variable. If it were made an instance variable of ParkingFacility, what would happen when the university runs two facilities (north and south gates)? Propose the solution for each case and analyse the space used by the counter for k facility objects and n tickets.

Evidence Expected

Correct and justified use of static members with an explanation of object passing and the object life cycle.

TASK 6 – Design the Inheritance Hierarchy 4 Marks**Bloom's Level:**

K4 – Analyse

Design an inheritance hierarchy for the vehicles and, if appropriate, for the users (student, staff, visitor).

Discuss:

- The superclass and subclasses, and the type of inheritance used (single, multilevel or hierarchical)
- What is inherited and what is added in each subclass
- How super is used for fields, methods and constructors
- The order in which constructors execute when an ElectricCar object is created, shown by a trace

Draw the hierarchy as a **UML class diagram**.

Alternative Concept and Complexity Analysis

You used inheritance (Car is-a Vehicle). If composition were used instead – a single Vehicle class that has-a VehicleType object holding the rate – what would the solution be? Compare the work needed to add a new vehicle type and analyse the time to compute a fee and the space per vehicle object in both designs.

Evidence Expected

A justified inheritance hierarchy, UML diagram, use of super and a traced constructor execution order.

TASK 7 – Override and Specialise Behaviour 4 Marks**Bloom's**

Level: K4 – Analyse

Identify the behaviour that must differ between subclasses and implement it by method overriding.

Discuss:

- Which methods are overridden (for example: calculateFee()) and why the subclass version is needed
- Use of the @Override annotation and the rules of overriding (signature, return type, access)
- Where the final keyword is appropriate for a variable, method or class, and why • How toString(), equals() and hashCode() are overridden so that two Vehicle objects with the same registration number

are treated as equal

Alternative Concept and Complexity Analysis

Each subclass overrides the fee method. If a single calculateFee() with a switch on a type field were used instead of overriding, what would the solution be? Analyse the time per fee calculation for k vehicle types and the number of classes that must change when a new type is added.

Evidence Expected

Correct overriding with @Override, justified use of final, and correct Object-class method overrides.

TASK 8 – Apply Polymorphism 4 Marks

Bloom's Level: K4 – Analyse

Demonstrate both compile-time and runtime polymorphism in your design.

Discuss:

- At least one overloaded method (for example: calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out)) and how the compiler selects it • How a single array of Vehicle references can hold different vehicle types (upcasting) • How dynamic method dispatch selects the correct calculateFee() at runtime • Why this design avoids long if–else chains that test the vehicle type

Trace one call through **dynamic method dispatch**, showing the declared type, the actual object type and the method executed.

Complete the Code

Complete the Vehicle hierarchy below and execute FeeDemo. It applies inheritance (Task 6), overriding (Task 7), polymorphism (Task 8) and an interface (Task 9).

```
interface Chargeable {
 double chargingFee(int kWh);
}

abstract class Vehicle {
 private final String regNo;

 protected Vehicle(String regNo) { this.regNo = regNo; }  public String

getRegNo() { return regNo; }

 protected abstract double ratePerHour();

 /** Every started hour is charged in full. */
 public double calculateFee(int minutes) {
 // TODO 1: Compute billable hours as the ceiling of minutes / 60 using integer //
arithmetic only, and return hours × ratePerHour()
 }

 @Override public boolean equals(Object o) {
 return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo);  }

 @Override public int hashCode() { return regNo.hashCode(); } }

class TwoWheeler extends Vehicle {
 TwoWheeler(String regNo) { super(regNo); }
 @Override protected double ratePerHour() { return 10; } }

class Car extends Vehicle {
 // TODO 2: Write the constructor using super, and override ratePerHour() to return
// 30

 }

class ElectricCar extends Car implements Chargeable {
ElectricCar(String regNo) { super(regNo); }
 // TODO 3: Implement chargingFee() at Rs 12 per kWh
 }

public class FeeDemo {
 public static void main(String[] args) {
 Vehicle[] parked = { new TwoWheeler("TN01A1"), new Car("TN02B2"),  new
ElectricCar("TN03C3") };  for (Vehicle v : parked) {
 double fee = v.calculateFee(130); // dynamic method dispatch if (v instanceof
Chargeable c) fee += c.chargingFee(10);
System.out.println(v.getClass().getSimpleName() + " " + v.getRegNo() + " -> Rs " +
fee);
 }
 System.out.println(new Car("TN02B2").equals(new ElectricCar("TN02B2"))); }
 }
```

Expected output:

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

Explain why the last line prints true, and decide whether this is the correct behaviour for your system.

Alternative Concept and Complexity Analysis

You used dynamic method dispatch. If the same behaviour were written using instanceof checks and casting for each vehicle type, what would the solution be? Analyse the time per call for k vehicle types (O(1) dispatch versus up to k checks) and the effect on code size and maintenance.

Evidence Expected

Working examples of overloading and dynamic dispatch with a traced call and an explanation of the design benefit.

TASK 9 – Introduce Abstraction 4 Marks

Bloom's Level: K4 – Analyse

Use abstract classes and interfaces to separate what the system does from how it is done.

Discuss:

- Which class should be abstract (for example: Vehicle) and which methods should be abstract
- Which capability should be an interface (for example: Chargeable for EVs, Payable for fee-paying users)
- Why you chose an abstract class in one place and an interface in another • How a class can obtain multiple behaviours through interfaces • Where a default or static method in an interface is useful

Alternative Concept and Complexity Analysis

Vehicle is an abstract class and Chargeable is an interface. If Vehicle were changed to an interface with default methods, and Chargeable to an abstract class, what would the solution be? Identify what can no longer be expressed (constructors, state, multiple inheritance) and analyse any difference in time per method call and space per object.

Evidence Expected

A justified use of at least one abstract class and one interface, with the reasoning behind each choice.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers 3 Marks

Bloom's Level: K4 – Analyse

Organise your classes into packages and protect the data of each class.

Discuss:

- Your package structure (for example: parking.model, parking.service, parking.app) • The access modifier chosen for each field and method (private, default, protected, public) and why

- How getters and setters validate data (for example: rejecting a negative duration or an invalid slot number)
- Where wrapper classes and autoboxing are used, and any risk they introduce (for example: comparing Integer objects with ==)

Alternative Concept and Complexity Analysis

Your fields are private with validating setters. If the fields were made public and validation were done by the calling code, what would the solution be and what could go wrong? Analyse the time cost of validation per update and the number of places where the same check would have to be repeated.

Evidence Expected

A package structure, justified access modifiers and validated getters and setters that protect the object state.

TASK 11 – Implement, Test and Defend the Design5 Marks

Bloom's Level: K5 – Evaluate

Implement the complete system in Java and test it.

Test and discuss:

- Normal operation: vehicles of different types entering and leaving • Boundary cases: the last free slot, zero-minute parking, a stay exactly at a fee-slab boundary
- Invalid cases: an unknown vehicle type, a duplicate registration number, an invalid slot number
- A small scenario whose total collection for the day you calculate by hand and compare with the program output
- How well your design follows encapsulation, inheritance, polymorphism and abstraction, and one limitation of it

Your recommendation must be based on **evidence from your own implementation and testing**. Simply stating that the program runs is not sufficient.

Alternative Concept and Complexity Analysis

Active tickets are currently stored in an array. If they were stored in an ArrayList<ParkingTicket> or a HashMap<String, ParkingTicket> keyed by registration number, what would the exit processing solution be? Measure and analyse the time to find a ticket on exit and the space used for n parked vehicles in each structure.

Evidence Expected

Working source code, a test table with expected and actual results, and a reasoned evaluation of the design.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks**Bloom's Level:** K5 – Evaluate

Changed Requirement

The university introduces **two new vehicle categories** – e-bikes (charged a flat fee) and delivery vans (charged at double the car rate during 9 AM – 5 PM). Visitors may now buy a **one-day pass**. The facility is extended from 3 to **5 floors** with a different number of slots on each floor.

Re-evaluate your original design in response to these changed requirements. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

Discuss:

• What has changed from the original problem • Which of your classes, methods and data structures are affected • Why a different number of slots on each floor affects your array design • Which classes, if any, would have to be modified rather than extended

Evidence Expected

A clear analysis of how the changed requirements affect the original design. **B.**

Evaluation of the Original Design – 2 Marks

Evaluate whether your original design remains appropriate. Discuss:

• Whether new vehicle types can be added only by writing new subclasses • Whether the pricing rules are spread across many classes or kept in one place • Whether the existing tested classes can remain unchanged

Evidence Expected

A technically supported evaluation of the original design under the new requirements. **C.**

Adaptation or Redesign – 2 Marks

Decide whether to retain, modify or redesign the design. If you modify or redesign it: • Describe the revised class structure (for example: a jagged array for floors, a Pass interface, a PricingRule abstraction)

• Explain what has changed

• Implement the new vehicle categories and the one-day pass

If you retain the original design, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response with clear object-oriented reasoning and working code. **D.**

Evidence-Based Defence – 2 Marks

Defend your final design decision using:

• The number of existing classes that had to change

• The ease of adding a further vehicle type in future

• Test results for the new categories

• Relevant trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating object-oriented principles, test evidence and the changed requirements.

MARK DISTRIBUTION

Task	**Bloom's Level**	**Marks**
Task 1 – Problem Analysis & Objects	K4 – Analyse	4
Task 2 – Data Representation	K4 – Analyse	3

Task 3 – Control Logic & Pseudocode	K4 – Analyse	4
Task 4 – Classes, Objects & Constructors	K4 – Analyse	4
Task 5 – static Members & Object Life Cycle	K4 – Analyse	3
Task 6 – Inheritance Hierarchy	K4 – Analyse	4
Task 7 – Method Overriding	K4 – Analyse	4
Task 8 – Polymorphism	K4 – Analyse	4
Task 9 – Abstraction	K4 – Analyse	4
Task 10 – Packages & Encapsulation	K4 – Analyse	3
Task 11 – Implementation, Testing & Defence	K5 – Evaluate	5
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UMLdiagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

SIMATS ENGINEERING

Course CSA0928 – Programming in Java

ASSIGNMENT 1

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

CO1 · Total marks 50 · Individual

Item	Details
Student name	**RAKSHITH R.R**
Register number	**192324156**
Department / Section	**B.Tech -AI&DS**
Java version used	**OpenJDK 21**
GitHub repository link	**192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA/ASSIGNMENT - 1 at main · RAKS-rak57/192324156---RAKSHITH-R.R---CSA0928---PROGRAMMING-IN-JAVA**
Submission date	**12/11/2026**

Overview, Repository Map and Task Summary

How this report is organised

Each task contains: the analysis asked for, the evidence expected, the **Complete the Code** result where applicable, and the **Alternative Concept and Complexity Analysis** with measurements from the executed programs. Every program was compiled and run with OpenJDK 21; the raw outputs are saved as text files and as screenshots in each task's output folder of the repository. Timings were taken on one machine with System.nanoTime() after warm-up; they are small micro-benchmarks, so only orders of magnitude and growth rates are reliable.

Repository map

```
CSA09-<RegisterNumber>-Assignment1/
├── README.md                    name, register number, Java version, how to compile/run each task, result of each task
├── run_all.sh                   compiles and runs every task
├── docs/uml/                    UML class diagrams (.dot source + .png)
├── Task01 … Task12/
│   ├── src/                     Java source code
│   ├── input/                   input files (where a task reads input)
│   └── output/                  output .txt and screenshots (.png)
├── Task07/negative, Task09/negative, Task10/negative   deliberately wrong code showing compiler errors
```

Task summary

Task	Topic	Marks	Main evidence (folder)	Result
1	Problem analysis, objects	4	Analysis; Task01/ parallel-array comparison	Done
2	Data representation	3	Task02/ casting bug demo, array comparison, Scanner input	Done
3	Control logic, pseudocode	4	Task03/ParkingFacility.java – output equals expected	Output identical
4	Classes, constructors	4	Task04/ParkingTicket.java	Done
5	static, life cycle	3	Task05/ counter, GC, passing objects	Done
6	Inheritance	4	Task06/ constructor trace, UML	Done
7	Overriding	4	Task07/ + compiler errors	Done

8	Polymorphism	4	Task08/FeeDemo.java – output equals expected	Output identical
9	Abstraction	4	Task09/ + compiler errors	Done
10	Packages, encapsulation	3	Task10/ packages, access, wrappers	Done
11	Implement, test, defend	5	Task11/ 14 classes, 35 tests	35 / 35 pass
12	Design challenge	8	Task12/ modified design, 35 regression + 31 new tests	35 / 35 and 31 / 31 pass

Complete-the-Code programs: ParkingFacility (Task 3) prints [0, 0] [0, 1] [1, 0] null true false false [0, 1]; FeeDemo (Task 8) prints Rs 30.0, Rs 90.0, Rs 210.0, true – both match the expected output exactly.

Task 1 – Understand the Problem and Identify the Objects (4 marks)

1.1 Inputs and outputs

Operation	Inputs received	Outputs produced
Facility set-up	Number of floors and the number of slots on each floor	A facility with every slot free
Vehicle entry	Registration number, vehicle type (two-wheeler / car / EV), user role (student / staff / visitor), pass status, entry time	A slot position (floor, slot), a ticket with a unique number, or a refusal (full / duplicate / unknown type)
Vehicle exit	Registration number, exit time, kWh charged (EVs only)	Receipt: minutes parked, parking fee, charging fee, total payable; the slot is freed
End of day	–	Vehicles inside, free slots, total collection

1.2 Nouns → candidate classes, verbs → methods

The problem statement was read sentence by sentence. Nouns that carry their own data or rules became candidate classes; verbs became methods of the class that owns the data they use.

Candidate class	Why it is needed (justification)	Responsibilities / main methods
Vehicle (abstract) + TwoWheeler, Car, ElectricCar	The three vehicle kinds differ in hourly rate and capabilities (only EVs charge) but share a registration number and the fee algorithm. One abstract parent holds the shared part; subclasses hold only what differs.	getRegNo(), ratePerHour(), calculateFee(minutes), equals()/hashCode() by registration number
User (abstract) + Student, Staff, Visitor	Passes are the only thing that depends on WHO parks: students and staff may hold a monthly pass, visitors always pay. Keeping this out of Vehicle means a car does not need to know who drives it.	isFeeExempt(date), role()

ParkingTicket	The ticket is the record that links vehicle, user, slot and entry time, and owns the unique number. Without it the exit step could not find the entry time.	Holds ticketNo, vehicle, user, floor, slot, entryTime; immutable
ParkingFacility	The floors and slots exist independently of any vehicle. Allocation and release are the only operations that touch the slot structure, so they live together.	allocate(), release(), freeSlots(), totalSlots()
ParkingService	Entry and exit are workflows that coordinate facility, ticket and fee. Putting them in ParkingFacility would mix storage with billing.	enter(), exit(), active-ticket lookup, running collection
Receipt	The result of an exit is a bundle of values that must not change after issue.	totalPayable()
VehicleFactory	Gate staff type a word ("CAR"); something must turn that into the right object and reject unknown words.	create(type, regNo)

Nouns deliberately NOT made classes: Floor and Slot. A slot holds only a registration number or null; an object per slot (500 objects for the largest facility) would add memory and indirection with no behaviour. If slots later gain attributes (EV charger, size, reserved) this decision should be revisited.

Verbs → methods: allocate a slot → ParkingFacility.allocate; release → release; issue a ticket → ParkingService.enter (constructs ParkingTicket); compute a fee → Vehicle.calculateFee; leave → ParkingService.exit.

1.3 Business rules every valid operation must satisfy

#	Rule	Where it is enforced
R1	A slot can hold at most one vehicle	allocate() only fills a slot that is null
R2	A ticket number is never reused	Class-wide counter incremented in the ParkingTicket constructor; a duplicate copy keeps the old number but is flagged (Task 4)
R3	A registration number can be inside the facility only once	ParkingService.enter → duplicate check
R4	Exit time may not be earlier than entry time	ParkingService.exit, Vehicle.calculateFee

R5	Every started hour is billed in full; a stay of 0 minutes is free	Vehicle.calculateFee
R6	A 24-hour block never costs more than 8 hours' worth (daily maximum)	Vehicle.calculateFee
R7	Students/staff with a pass valid on the entry date pay no parking fee; visitors pay in full	User.isFeeExempt
R8	Charging is billed separately, is never waived by a pass, and only EVs can be charged	ParkingService.exit, Chargeable
R9	The slot is freed at exit and can be reused immediately	ParkingFacility.release
R10	Releasing an invalid or already-free position is rejected	release() returns false
R11	The vehicle type must be one of the known types	VehicleFactory.create

1.4 Assumptions about the facility

- Currency is rupees; hourly rates are Rs 10 (two-wheeler), Rs 30 (car and EV); charging is Rs 12 per kWh; daily maximum is 8 hours' worth.
- Time is recorded to the minute; the pass is checked against the **entry** date.
- First-fit allocation: the lowest floor and the lowest slot number that is free.
- There is one gate (one ParkingService); two gates are discussed in Task 5.
- Charging slots are not a separate pool: charging is billed from the kWh reading entered at exit.
- Registration numbers consist of 4–12 capital letters or digits.

1.5 UML class diagram of the identified classes

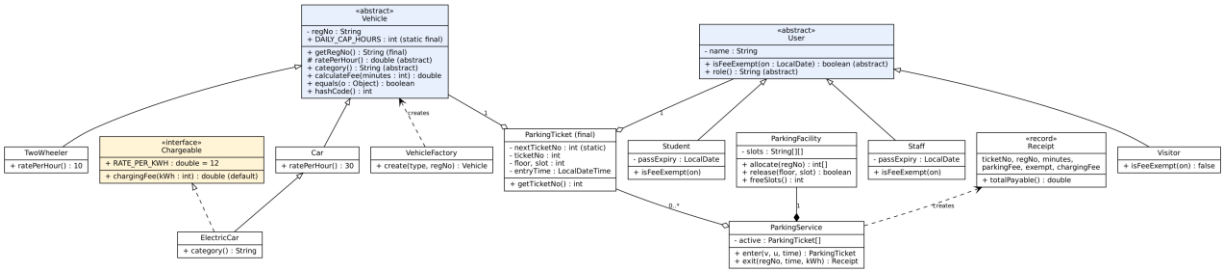


Figure 1.1 – Candidate classes and their relationships (from the implemented design)

1.6 Alternative Concept and Complexity Analysis – plain Strings in parallel arrays

What the solution would look like. Instead of one Vehicle object per vehicle, three arrays are kept in step: regNo[], type[], entryTime[]. Index i in all three describes the same vehicle.

```
String[] regNo = new String[n];   String[] type = new String[n];   long[] entryTime = new long[n];
int find(String reg) {                    // search by registration number
   for (int i = 0; i < n; i++) if (regNo[i].equals(reg)) return i;
   return -1;
}
double fee(int i, int minutes) {          // behaviour must be selected by testing the String
   if (type[i].equals("CAR")) return ((minutes + 59) / 60) * 30;
   else if (type[i].equals("TWO")) return ((minutes + 59) / 60) * 10;
   ...
}
```

Criterion	Objects (`Vehicle[]`)	Parallel arrays
Find by registration number	Linear scan, O(n): v[i].regNo.equals(reg) – one extra pointer to follow per element	Linear scan, O(n): regNo[i].equals(reg) – the strings are in one contiguous array of references
Add a new attribute (e.g. owner)	Add ONE field to the class; existing code untouched	Add a 4th array AND update every add / remove / shift / copy routine so that all k+1 arrays stay aligned – one forgotten array silently corrupts the data
Behaviour that depends on the type	Method overriding (Tasks 7–8)	if/switch on the type[] string in every function; typos are not caught by the compiler
Space per vehicle (container only)	≈ 32 bytes measured (object header + 3 references/long + padding)	≈ 16 bytes measured (2 references + 1 long)

Measured worst-case search (target is the last element), nanoseconds per search:

n (vehicles)	Objects (ns)	Parallel arrays (ns)
1,000	2478	2483
10,000	104857	43316
100,000	422593	213921

Analysis. Both searches are Θ(n) time, so the choice does not change the asymptotic cost: a 10× larger n gives a 20–40× longer search in the table (cache effects add to the linear growth). In this run the parallel arrays were equal at n = 1,000 and up to about twice as fast at larger n, because each step reads a contiguous array of references instead of following one more pointer to an object; timings on a shared machine are noisy, so only the order of growth is reliable. The objects cost about 16 more bytes per vehicle (header and extra fields) and in exchange give O(1) behaviour dispatch, a single place to add attributes and compiler-checked types. For this system (at most 500 vehicles) memory and a few microseconds are irrelevant and maintainability decides: the object design is retained. Measured data come from Task01/output/Task01Demo.txt.

```
$ java -cp out Task01Demo
=== Task 1 : objects vs parallel arrays ===
n          objects (ns/search)    parallel (ns/search)
1000       2478                   2483                   (sink=1)
10000      104857                 43316                  (sink=1)
100000     422593                 213921                 (sink=1)


Measured container memory per vehicle: objects ~ 32 bytes, parallel arrays ~ 16 bytes
(objects: 12-16 byte header + 2 refs + long + owner ref, padded;  parallel: 2 refs + 1 long per index)
keep-alive: 200000 200000 200000 200000


Adding attribute 'owner':
 objects  -> add one field to VehicleRecord (1 place changed, existing code untouched)
 parallel -> add a 4th array AND update every add/remove/copy/shift routine (k+1 arrays kept in sync)
 objects  demo: objs[5].owner = Priya
```

Task 2 – Choose the Data Representation (3 marks)

2.1 Data type for each field

Field	Type	Reason
Slot number, floor number	int	Small whole numbers used as array indices (indices must be int)
Vehicle registration number	String	Letters and digits (TN01AB1111); never used in arithmetic, so not a number; compared with equals()
Entry / exit time	java.time.LocalDateTime	Immutable, minute arithmetic via Duration, handles midnight and month ends. long epoch milliseconds would also work but an (int) cast of it overflows (shown below)
Duration	int (minutes)	A stay is at most days (10,080 minutes a week); int holds 2.1 billion; whole minutes are the billing unit
Fee, rate, charging fee	double	Mixed with the rate in expressions; rates here are whole rupees so every value is exactly representable. For fractional money BigDecimal or int paise would be required (0.1 + 0.2 = 0.30000000000000004)
kWh drawn	int	Whole units read from the charger
Ticket number	int	Counter starting at 1000
Pass flag / exempt	boolean / LocalDate expiry	Yes/no decisions; expiry compared by date

2.2 Representing floors and slots

A **jagged array** String[][] slots was chosen: slots[f] is the row for floor f and has its own length. slots[f][s] holds a registration number or null (free). Reasons: (a) the facility has floors of different sizes (Task 12 uses 20, 35, 50, 75, 100); a rectangular String[floors][maxSlots] would waste memory and need a separate length per floor anyway; (b) lookup slots[f][s] is O(1); (c) slots[f].length gives the size of every floor with no extra field.

2.3 Where type conversion occurs

Place	Conversion	Risk and resolution

Minutes → billable hours	int / int	Integer division discards the remainder. Solved with (minutes + 59) / 60, a ceiling computed entirely in integers
Duration.toMinutes() → int	long → int (narrowing)	Could overflow only for stays longer than 4,000 years; exit before entry is rejected first
hours * ratePerHour()	int → double (widening)	Safe; no information lost
Epoch millis long → int	narrowing	Would give garbage; avoided by storing LocalDateTime

Example: integer division and wrong casts produce a wrong fee (Car, Rs 30/hour)

```
=== Part 2: casting risks (Car, Rs 30/hour) ===
minutes  int-division late cast    truncating     correct    ceil(dbl)
1        0.0         0.0         0.0           30.0       30.0
59       0.0         0.0         0.0           30.0       30.0
60       30.0        30.0        30.0          30.0       30.0
61       30.0        30.0        30.0          60.0       60.0
130      60.0        60.0        60.0          90.0       90.0
150      60.0        60.0        60.0          90.0       90.0
-> 130 minutes: int division bills Rs 60 instead of Rs 90 (loses Rs 30 = 33% of the fee).
long->int narrowing: 1762500000000 becomes 1563408640  (overflow) -> keep timestamps in long/LocalDateTime
double money risk: 0.1 + 0.2 = 0.30000000000000004  -> fees are whole rupees; use int paise or BigDecimal for fractional money
widening is safe: int hours = 3; double fee = hours * 30.5 = 91.5
```

For a **130-minute** stay the integer-division version bills 130 / 60 = 2 hours = **Rs 60** instead of the correct 3 started hours = **Rs 90** – the facility loses Rs 30 (33 %). A stay of 1–59 minutes is billed **Rs 0**. Casting afterwards, (double)(minutes / 60), does not help because the division has already happened; (int)(minutes / 60.0) truncates 2.17 to 2. The correct versions are (minutes + 59) / 60 or Math.ceil(minutes / 60.0) (the cast/divide happens in double first). The implementation uses the integer form so that no floating-point value is involved in counting hours.

2.4 How input is read

Scanner on System.in is used (Task02Demo, and Main in Tasks 11–12). Input is redirected from a file (java Task02Demo < input/input.txt), so a whole day of events can be replayed repeatedly and compared with the hand calculation. Command-line arguments (args[]) were rejected because they suit one value per run, not an ordered list of entry/exit events; interactive typing is not repeatable. nextInt()/next() are used for tokens; nextLine() plus split in Main so that blank and comment lines can be skipped.

```
4
TN01AB1111 TWO 45
TN02CD2222 CAR 130
TN03EF3333 CAR 60
TN04GH4444 CAR 61
```

```
$ java -cp out Task02Demo < input/input.txt
=== Part 1: reading input with Scanner (input/input.txt) ===
TN01AB1111   TWO    45 min  correct fee = Rs 10.0
TN02CD2222   CAR   130 min  correct fee = Rs 90.0
TN03EF3333   CAR    60 min  correct fee = Rs 30.0
TN04GH4444   CAR    61 min  correct fee = Rs 60.0


... (21 more lines in the .txt file)
```

2.5 Alternative Concept and Complexity Analysis – 1-D array or ArrayList<String[]>

Flat 1-D array. index = floor × slotsPerFloor + slot. This formula only works when every floor has the same size. Demonstrated in the program: with floors {20, 35, 50, 75, 100}, slot 40 of floor 3 is at index 20+35+50+40 = 145, but the formula with 20 slots per floor gives 3×20+40 = 100, which is another slot (printed as null in the output). With uneven floors an **offset array** offset[f] (prefix sums, offset[f+1] = offset[f] + size[f]) is needed, and index = offset[f] + slot; the offset table must be rebuilt whenever a floor is added.

ArrayList of rows. list.get(f)[s] – floors may be added at run time (add) without copying the whole structure, and each element is a row of its own size, so uneven floors need no special handling.

Representation	Locate slot (time)	Space	Uneven floors	Measured ns/lookup

Jagged String[][]	O(1)	F row objects + Σ sizes references	Natural	1.43
ArrayList<String[]>	O(1) (get + index)	As jagged + list backing array (capacity ≥ F)	Natural; floors can be added	2.16
Flat String[] + offset[]	O(1) with the offset table (O(F) or O(log F) if the offset has to be searched)	1 array of Σ sizes + offset[F+1]; fewest objects, best cache locality	Needs offset table; naive formula is wrong	1.54

Conclusion. All three give O(1) access; the measured differences (1–2 ns) are irrelevant at this scale. The jagged array is retained because it is the simplest correct structure for uneven floors and it kept the Task 12 change (3 → 5 floors with different sizes) to zero lines of code in ParkingFacility.

Task 3 – Design the Control Logic (4 marks)

3.1 How slots are searched, and when the search stops

allocate is a **first-fit** search: an outer for loop over floors and an inner for loop over the slots of the current floor. The first null slot found is filled and return {floor, slot} leaves **both loops at once**, so no further slots are examined. If both loops finish without finding a free slot the method returns null (facility full); the caller turns that into a refusal.

3.2 Fee slabs with if–else / switch

Three slabs are applied in this order: (1) zero minutes → free; (2) every **started** hour at the hourly rate (first hour and additional hours cost the same in the base design); (3) a **daily maximum** of 8 hours' worth for every 24-hour block. The rate comes from a switch on the vehicle type (the program in the repository uses switch with break/return; the final system replaces it with overriding, Tasks 7–8).

Minutes	Two-wheeler	Car / EV	Slab reached
0	0	0	free
1, 59, 60	10	30	first hour
61, 120	20	60	second hour
130	30	90	3 started hours
480 – 1439	80	240	daily maximum (8 h)
1440	80	240	one full day = one cap
1441	90	270	cap + 1 started hour

3.3 Use of break, continue and return

- return inside the nested loops of allocate – leaves both loops the moment a slot is placed (a break would leave only the inner loop and would need a flag).
- Early return 0 for a zero-minute stay in the fee method – no division or loop is needed.
- break at the end of each case in the type switch – without it execution would fall through to the next rate.
- continue in Main to skip blank and # comment lines of the input file without nesting the whole body in an if.
- break in the lookup loops of the benchmark once the registration number is found.
- In the count-plus-stack alternative, a floor whose free counter is 0 is skipped (continue semantics) without scanning its slots.

3.4 When no suitable slot is free

ParkingFacility.allocate returns null; ParkingService.enter throws IllegalStateException("Parking facility is full") **before** a ticket is created, so no ticket number is consumed and no state changes. Tested in Task 11 (test 11).

3.5 Pseudocode

The pseudocode below uses a fixed layout (ALGORITHM / INPUT / OUTPUT / BEGIN … END, ← for assignment, upper-case keywords, indentation for nesting). **Note to the student:** align the keywords and layout with the *SIMATS Engineering – Standard for Algorithmic Pseudocode* document supplied by your faculty before submitting – the logic stays the same.

```
ALGORITHM AllocateSlot
INPUT  : vehicleNo : text;  slots[0..F-1][0..S(f)-1] : table of text or NULL
OUTPUT : the pair (floor, slot) that now holds vehicleNo, or NULL when full
BEGIN
   FOR floor ← 0 TO F-1 DO
      FOR slot ← 0 TO LENGTH(slots[floor]) - 1 DO
         IF slots[floor][slot] = NULL THEN
            slots[floor][slot] ← vehicleNo
            RETURN (floor, slot)          // leaves both loops
         END IF
      END FOR
   END FOR
   RETURN NULL                           // every slot is occupied
END


ALGORITHM ReleaseSlot
INPUT  : floor, slot : integers
OUTPUT : TRUE if a vehicle was removed, otherwise FALSE
BEGIN
   IF floor < 0 OR floor ≥ F THEN RETURN FALSE
   IF slot < 0 OR slot ≥ LENGTH(slots[floor]) THEN RETURN FALSE
   IF slots[floor][slot] = NULL THEN RETURN FALSE      // already free
   slots[floor][slot] ← NULL
   RETURN TRUE
END
```

```
ALGORITHM ComputeFee
INPUT  : minutes : integer ≥ 0;  rate : rupees per hour
OUTPUT : fee in rupees
CONSTANT MINUTES_PER_DAY ← 1440;  DAILY_CAP_HOURS ← 8
BEGIN
   IF minutes < 0 THEN RAISE error "negative duration"
   IF minutes = 0 THEN RETURN 0                    // slab 1: free
   cap       ← DAILY_CAP_HOURS × rate
   fullDays  ← minutes DIV MINUTES_PER_DAY          // integer division
   remainder ← minutes MOD MINUTES_PER_DAY
   hours     ← (remainder + 59) DIV 60              // ceiling of remainder/60, integers only
   remainderFee ← hours × rate                      // slab 2: every started hour
   IF remainderFee > cap THEN remainderFee ← cap    // slab 3: daily maximum
   RETURN fullDays × cap + remainderFee
END
```

3.6 Complete the Code – ParkingFacility

The program was copied into Task03/src/ParkingFacility.java, both TODOs were replaced and main() was left untouched.

```
public class ParkingFacility {
   // slots[floor][slot] holds a vehicle number, or null if the slot is free
   private final String[][] slots;


   public ParkingFacility(int[] slotsPerFloor) {
      slots = new String[slotsPerFloor.length][];
      for (int f = 0; f < slotsPerFloor.length; f++) {
         slots[f] = new String[slotsPerFloor[f]];
      }
   }


   /** Returns {floor, slot} of the first free slot, or null if it is full. */
   public int[] allocate(String vehicleNo) {
      // TODO 1: Search floor by floor and slot by slot; place vehicleNo in the first
      // free slot and return {floor, slot}
      for (int f = 0; f < slots.length; f++) {
```

```
        for (int s = 0; s < slots[f].length; s++) {
            if (slots[f][s] == null) {
                slots[f][s] = vehicleNo;
                return new int[]{f, s};      // return stops BOTH loops at once
            }
        }
    }
    return null;
}


/** Frees a slot. Returns false if the position is invalid or already free. */
public boolean release(int floor, int slot) {
    // TODO 2: Validate floor and slot, free the slot and return true; otherwise
    // return false
    if (floor < 0 || floor >= slots.length) return false;
    if (slot < 0 || slot >= slots[floor].length) return false;
    if (slots[floor][slot] == null) return false;
    slots[floor][slot] = null;
    return true;
}


public static void main(String[] args) {
    // floor 0 has 2 slots, floor 1 has 1 slot
    ParkingFacility p = new ParkingFacility(new int[]{2, 1});
    System.out.println(java.util.Arrays.toString(p.allocate("TN01AB1111")));
    System.out.println(java.util.Arrays.toString(p.allocate("TN02CD2222")));
    System.out.println(java.util.Arrays.toString(p.allocate("TN03EF3333")));
    System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
    System.out.println(p.release(0, 1) + " " + p.release(0, 1) + " "
            + p.release(5, 0));
    System.out.println(java.util.Arrays.toString(p.allocate("TN04GH4444")));
}
}
```

Expected output	Actual output
[0, 0]	[0, 0]

[0, 1] [1, 0] null true false false [0, 1]	[0, 1] [1, 0] null true false false [0, 1]

The two outputs are identical. Reading the output: the three vehicles fill [0,0], [0,1], [1,0]; the fourth finds the facility full (null); release(0,1) succeeds (true), the second release(0,1) fails because the slot is already free, release(5,0) fails because floor 5 does not exist; the final allocate therefore reuses the freed slot [0,1].

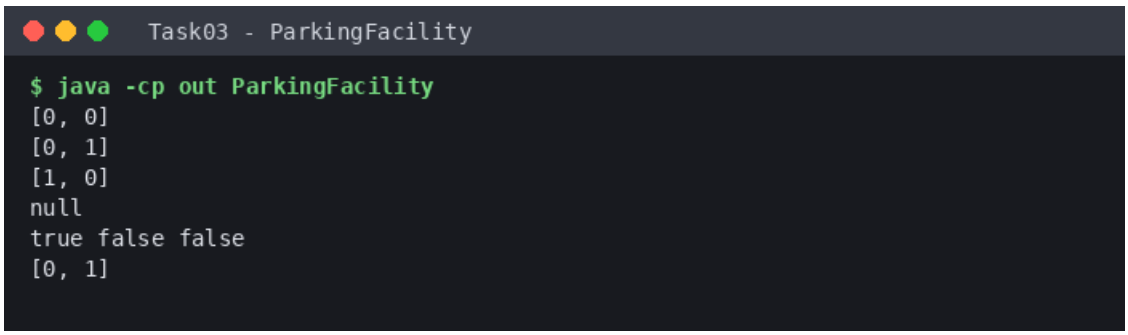


Figure 3.1 – Output of ParkingFacility (from Task03/output)

3.7 Alternative Concept and Complexity Analysis – free count per floor + stack of free slots

Solution. Each floor keeps freeCount[f] and a stack freeStack[f] of its free slot numbers (initially pushed so that slot 0 is on top). allocate looks at the counters only, then pops; release validates, clears the slot and pushes the number back (FastParkingFacility.java).

```
ALGORITHM AllocateSlot_Fast
BEGIN
  FOR floor ← 0 TO F-1 DO
    IF freeCount[floor] = 0 THEN CONTINUE          // skip a full floor without scanning it
    slot ← POP(freeStack[floor])
    slots[floor][slot] ← vehicleNo;  freeCount[floor] ← freeCount[floor] - 1
    RETURN (floor, slot)
  END FOR
  RETURN NULL
END
ALGORITHM ReleaseSlot_Fast   // validate as before, then
```

```
  slots[floor][slot] ← NULL;  PUSH(freeStack[floor], slot);  freeCount[floor] ← freeCount[floor] +
1
```

Operation	**Scanning design**	**Count + stack design**
allocate() worst case	O(F · S) – every slot examined	O(F) – only the F counters; the pop is O(1). O(1) if one global free-stack is used
allocate() best case	O(1) (slot [0,0] free)	O(1)
release()	O(1)	O(1)
Extra space	none	F counters + one stack entry per free slot: O(F + F·S) = O(F·S) extra integers

Measured: all slots occupied except the last one (worst case for the scan), time for one allocate + release:

Facility	**Slots**	**Scan (ns)**	**Count + stack (ns)**
1 floor x 20 slots	20	6.1	7.6
3 floors x 50 slots	150	38.3	6.3
5 floors x 100 slots	500	220.8	5.6
20 floors x 500 slots	10,000	2111.9	24.6
50 floors x 2000 slots	100,000	23937.8	33.7

Analysis. For the three sizes in the scenario (20, 150 and 500 slots) the scan costs 7–230 ns, which is negligible, but the scan grows linearly with F·S (tens of microseconds at 100,000 slots) while the stack version stays at a few nanoseconds. One consequence of the stack design: after several exits, slots come back in last-in-first-out order, so "first free slot" is no longer guaranteed to be the lowest-numbered one unless a priority queue (O(log S) per operation) is used. The scan was retained for the base system because the facility is small and the rule "lowest free slot first" falls out naturally; the stack version is the recommended upgrade if the facility grows.

```
$ java -cp out FeeCalculator
minutes  TWO_WHEELER  CAR      EV
0        0.0          0.0      0.0
1        10.0         30.0     30.0
59       10.0         30.0     30.0
60       10.0         30.0     30.0
61       20.0         60.0     60.0
```

```
120      20.0        60.0     60.0
130      30.0        90.0     90.0
480      80.0        240.0    240.0
481      80.0        240.0    240.0
720      80.0        240.0    240.0
1439     80.0        240.0    240.0
1440     80.0        240.0    240.0
1441     90.0        270.0    270.0
2970     180.0       540.0    540.0
BUS -> Unknown vehicle type: BUS
```

Task 4 – Design Classes, Objects and Constructors (4 marks)

4.1 Fields and methods

ParkingTicket (the class used to demonstrate constructors; the version used in the final system is in Task11/src/parking/model):

Member	Details
Fields	private static int nextTicketNo; private final int ticketNo, floor, slot; private final String regNo; private final LocalDateTime entryTime; private final boolean duplicate
Methods	Getters for every field, isDuplicate(), toString() – no setters: a ticket never changes after it is issued

4.2 Constructors and when each is used

Constructor	Use
ParkingTicket() – default	Creates an UNASSIGNED placeholder (ticketNo −1, floor/slot −1) that skips validation and does not consume a ticket number; used for pre-filling pools. Never issued to a vehicle.
ParkingTicket(regNo, floor, slot)	Normal entry at the gate: entry time is "now". Does **no work itself**: this(regNo, floor, slot, LocalDateTime.now())
ParkingTicket(regNo, floor, slot, entryTime)	The full constructor: validates all four values, takes the next ticket number. It is the only place where a normal ticket is built, so every route into the class passes through the same checks. Also used for replaying a stored day.
ParkingTicket(ParkingTicket other) – copy	Re-issues a duplicate ticket for a customer who lost the paper copy.

4.3 `this` and constructor chaining

```
public ParkingTicket(String regNo, int floor, int slot) {
    this(regNo, floor, slot, LocalDateTime.now());   // chaining: must be the FIRST statement
}
public ParkingTicket(String regNo, int floor, int slot, LocalDateTime entryTime) {
    if (regNo == null || regNo.isBlank()) throw new IllegalArgumentException("regNo required");
    ...
    this.ticketNo = nextTicketNo++;                  // 'this.' separates the field from the parameter
```

```
    this.regNo = regNo;
}
```

Chaining avoids repeating the validation and numbering code in each constructor: a rule changed once in the full constructor applies to every way of creating a ticket. this.x = x is needed because the parameter has the same name as the field.

4.4 Is a copy constructor needed?

Yes, in one place: **re-issuing a duplicate ticket**. ParkingTicket dup = new ParkingTicket(a) creates a second, independent object with the **same ticket number** (the number is *not* consumed again, so rule R2 "never reused" still holds – the number identifies one parking visit) and duplicate = true, so the cashier can see it is a copy. Writing ParkingTicket alias = a; instead would copy only the reference: two names for one object. The demonstration prints dup == a : false and alias == a : true. LocalDateTime is immutable, so the copy can safely share it; a mutable field would have to be copied too (deep copy).

4.5 How objects are stored in memory

```
STACK (per method call)            HEAP (shared, garbage collected)
 a          ──────────────────────────────────►   ParkingTicket#1000   {regNo  ─►
"TN01AB1111", floor 0, slot 0,
 alias      ──────────────────────────────────►        entryTime  ─►  LocalDateTime,
duplicate=false}
 dup        ──────────────────────────────────►  ParkingTicket#1000  {…, duplicate=true}
(different object)


STATIC AREA (class-wide, one copy)
 ParkingTicket.nextTicketNo = 1003
```

Reference variables (a, alias, dup) live on the stack of the running method; the objects they refer to live on the heap. Primitive fields (ticketNo, floor) are stored inside their object; String and LocalDateTime fields are references to further heap objects. The static counter belongs to the class, not to any ticket.

```
$ java -cp out Task04Demo
=== Constructors ===
a     = Ticket#1000[TN01AB1111 F0-S0 2026-11-07T09:30]
b     = TN02CD2222 ticketNo 1001
blank = Ticket#-1[UNASSIGNED F-1-S-1 2000-01-01T00:00]
```

```
=== Copy constructor (duplicate ticket) vs reference copy ===
dup   = Ticket#1000[TN01AB1111 F0-S0 2026-11-07T09:30 DUPLICATE]
dup == a      : false   (different objects)
alias == a    : true   (same object, two references)
same ticketNo : true   (number is not consumed again)
next new ticket number = 1002   (a=1000, b=1001, never reused)

=== Validation in the constructor ===
blank regNo   -> regNo required
negative floor -> floor/slot must be >= 0

=== Alternative 1: default constructor + setters ===
right after new: isValid() = false  (an INVALID object already exists)
after 4 setters : isValid() = true

=== Alternative 2: static factory ===
```

4.6 Alternative Concept and Complexity Analysis – default constructor + setters, or static factories

Setters. TicketSetterStyle t = new TicketSetterStyle(); t.setRegNo(..); t.setFloor(..); t.setSlot(..); t.setEntryTime(..); **Static factory.** TicketFactory.forEntry(reg, floor, slot, time) with a private constructor (also forEntryNow(...)). Both are in Task04/src.

Criterion	**Overloaded constructors + chaining**	**Default constructor + setters**	**Static factory methods**
Valid object guaranteed?	Yes – an object cannot exist without passing the checks; fields can be final (immutable)	No – the object exists in an invalid state after new (measured: isValid() = false); fields cannot be final; any setter can be forgotten	Yes – constructor is private, factory validates; fields final
Readability	Parameter order must be remembered	Self-describing calls but verbose	Named methods (forEntryNow) – the clearest at call sites

Different return type / caching	Not possible	–	Possible (could return a cached or subclass object)

Time to create n tickets (ms, best of 4 runs):

n	**Constructor chain**	**Default + 4 setters**	**Static factory**
100,000	3.5	7.1	2.9
1,000,000	17.8	17.7	13.1
5,000,000	30.5	26.2	26.4

Analysis. Creating n objects is Θ(n) time and Θ(n) space in all three approaches (each ticket is one heap object of the same fields). The setter style performs four extra method calls per object and the factory one extra static call; the measured differences (a few milliseconds, with the first row dominated by JIT warm-up) are within the noise of a shared machine and are not meaningful. The decision rests on **correctness, not speed**: the constructor/factory styles make an invalid ticket impossible, which the setter style cannot. Constructors with chaining were kept; a static factory would be a good next step if more ways of creating tickets were needed.

Task 5 – Apply static Members and the Object Life Cycle (3 marks)

5.1 Where static is needed

Member	Why static	Why an instance variable would be wrong
ParkingTicket.nextTicketNo (counter)	ONE counter must serve every ticket ever created	Each ticket would start its own counter at 1000 and every ticket would be numbered 1000 – rule R2 broken
ParkingTicket.issuedCount	Number of tickets created by the class as a whole	A per-object count is always 1
ParkingTicket.isValidRegNo(String) – static method	Pure utility that needs no ticket object; callable before any ticket exists	–
static { nextTicketNo = 1000; … } – static block	Runs once when the class is loaded: the right place for one-time set-up of static state (printed in the output)	A constructor would run for every ticket
Vehicle.DAILY_CAP_HOURS, Chargeable.RATE_PER_KWH (static final)	Business constants shared by all objects	One copy per object wastes memory and could be changed per object

A static method cannot use this or instance fields (no object is implied); it may use other static members. That is why getIssuedCount() is static but getTicketNo() is not.

5.2 Life cycle of a ParkingTicket

> new ParkingTicket(...) → object created on the heap, reference stored in active[i] (vehicle enters)
> active[i] used by exit() → fee computed from ticket.getEntryTime() (vehicle leaves)
> active[i] = active[--count]; active[count] = null → the ONLY strong reference is removed
> no reference reachable → ticket is ELIGIBLE for garbage collection
> garbage collector runs → memory reclaimed at a time the JVM chooses (never at a time the program can rely on)

The program demonstrates this with a WeakReference: after the reference in the array is set to null and System.gc() is requested, probe.get() == null shows that the ticket was collected (System.gc() is only a request; on the machine used the object was collected). Note the line active[count] = null in ParkingService.exit – without it the array would keep the old ticket alive and it could never be collected (a memory leak).

5.3 How objects are passed and returned

Java passes everything **by value**; for objects the value is a copy of the *reference*. Consequences shown in the output:

- reassign(t) assigns a new ticket to its parameter: only the method's local copy of the reference changes, the caller's t still points to the original (output: ticket unchanged).
- tamperArray(arr) changes arr[0] through the copied reference: the caller sees the change because both references point to the same array object.
- tamperInt(x) receives a copy of a primitive: the caller's x stays 5.
- A method can **return** an object (makeTicket, ParkingService.enter, exit returning a Receipt) – the caller receives the reference; nothing is copied. Immutable objects (ParkingTicket, Receipt) make this safe: the receiver cannot change them.

```
$ java -cp out Task05Demo
=== 1. Instance counter with two facilities (WRONG) ===
NORTH -> ticket 1000 for TN01AA1111
SOUTH -> ticket 1000 for TN02BB2222
NORTH -> ticket 1001 for TN03CC3333
SOUTH -> ticket 1001 for TN04DD4444
-> NORTH and SOUTH both issued 1000 and 1001: ticket numbers are reused across the campus.

=== 2. Static counter (shared by all facilities) ===
[static block] ParkingTicket class loaded, counter starts at 1000
NORTH -> ticket 1000 for TN01AA1111
SOUTH -> ticket 1001 for TN02BB2222
NORTH -> ticket 1002 for TN03CC3333
SOUTH -> ticket 1003 for TN04DD4444
tickets issued so far (static getIssuedCount) = 4

=== 3. Alternative: injected shared TicketIssuer ===
NORTH -> ticket 1000 for TN01AA1111
SOUTH -> ticket 1001 for TN02BB2222
```

```
NORTH -> ticket 1002 for TN03CC3333

=== 4. Static method ===
isValidRegNo("TN01AB1111") = true
isValidRegNo("hello")      = false

=== 5. Passing and returning objects ===
returned from method : Ticket#1004[TN07XY7777]
after reassign(t)    : Ticket#1004[TN07XY7777]   (caller's reference unchanged: Java passes the reference BY VALUE)
array after tamper   : CHANGED   (the shared object was modified)
int after tamperInt  : 5   (primitive copied)

=== 6. Life cycle: ticket becomes garbage after exit ===
vehicle inside   : ticket reachable = true
after exit + GC  : ticket collected = true   (eligible once no strong reference remains; collection timing is up to the JVM)

=== 7. Space used by the counter ===
static  counter: 1 int for the whole class, O(1) for k facilities
instance counter: 1 int per facility object, O(k)
tickets (n): O(n) in both designs
```

5.4 Alternative Concept and Complexity Analysis – counter as an instance variable of ParkingFacility

What happens with two facilities (north and south gates). Each ParkingFacility object would own its own counter starting at 1000, so NORTH and SOUTH both issue ticket 1000, 1001, … – the output of section 1 of the program shows ticket numbers repeated across the campus (rule R2 broken: two different vehicles hold the same number).

Solutions.

- **Static counter** (used in the final system): one counter shared by all facilities; numbers are unique campus-wide. Weakness: global mutable state, not thread-safe (would need AtomicInteger if gates ran in parallel), hard to reset in tests.

- **Injected issuer object:** one TicketIssuer created once and passed to both facilities (new FacilityWithIssuer("NORTH", issuer)); unique numbers without static state, testable. This is the cleaner design if more gates are added.
- **Per-facility counters with a prefix** ("N-1000", "S-1000"): numbers unique because the prefix is part of the number; each gate works independently.

Design	**Counter space for k facilities**	**Space for n tickets**	**Time to issue a number**
Static counter	O(1) – one int for the whole class	O(n)	O(1)
Instance counter (per facility)	O(k) – one int per facility object	O(n)	O(1)
Injected issuer	O(1) – one issuer object	O(n)	O(1)

The memory difference (a few bytes) is irrelevant; the design choice is about **correctness of uniqueness**, not space.

Task 6 – Design the Inheritance Hierarchy (4 marks)

6.1 Hierarchy and type of inheritance

Vehicle is the abstract superclass; TwoWheeler and Car extend it (**hierarchical** inheritance – several subclasses of one parent) and ElectricCar extends Car (**multilevel** – a chain of three levels: Vehicle → Car → ElectricCar). ElectricCar also implements the interface Chargeable. The users form a second hierarchical tree: User → Student, Staff, Visitor.

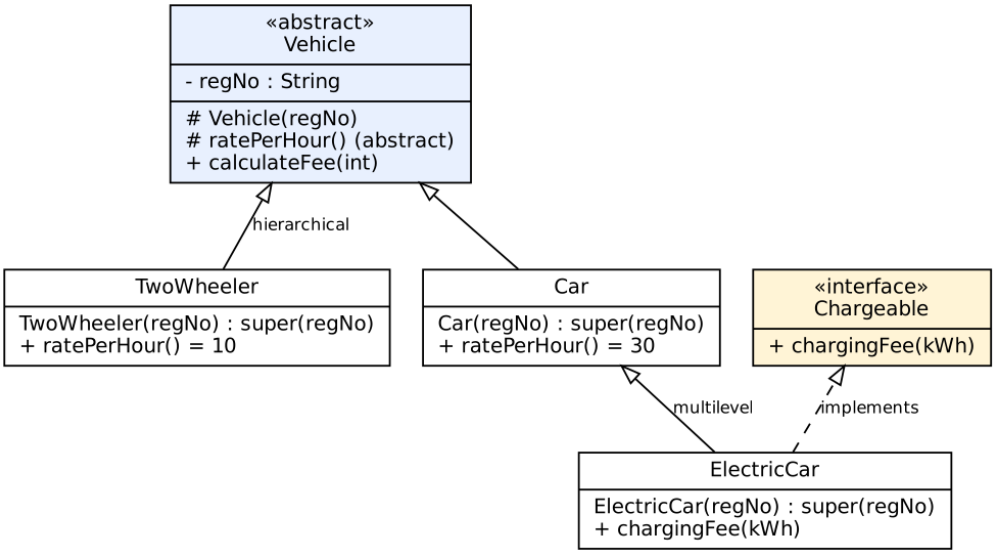


Figure 6.1 – UML class diagram of the vehicle hierarchy (solid arrow with hollow head = extends, dashed = implements)

6.2 What is inherited and what is added

Class	Inherited	Added / specialised
Vehicle (abstract)	–	private regNo, getRegNo(), abstract ratePerHour(), calculateFee(), equals/hashCode
TwoWheeler	regNo, getRegNo(), calculateFee()	ratePerHour() = 10
Car	same	ratePerHour() = 30
ElectricCar	everything from Car (incl. ratePerHour() = 30)	chargingFee(kWh) from Chargeable; category()
Student, Staff, Visitor	name, getName()	isFeeExempt(date) and role() implemented differently

6.3 How `super` is used

- **Constructors:** Car(String r) { super(r); } – calls Vehicle(String), the only way to set the private regNo. super(...) must be the first statement; if omitted, Java inserts super(), which would not compile because Vehicle has no no-argument constructor.
- **Methods:** ElectricCar.describe() returns super.describe() + " (EV, 40 kWh)" – reuses the parent's text and extends it (Task06Demo).
- **Fields:** regNo is private, so subclasses cannot use super.regNo; they go through the public getter. (Protected fields were avoided to keep encapsulation.)

6.4 Constructor execution order for `new ElectricCar("TN09EV0001", 40)`

```
=== Constructor trace: new ElectricCar("TN09EV0001", 40) ===
 [1] Vehicle constructor     : regNo = TN09EV0001
 [2] Car constructor
 [3] ElectricCar constructor : batteryKwh = 40
 -> order of execution: Vehicle (top) -> Car -> ElectricCar (bottom)
```

```
new ElectricCar("TN09EV0001", 40)
 1. ElectricCar constructor is entered; its first statement is super(r)        → control goes UP
 2.   Car constructor is entered; its first statement is super(r)              → control goes UP
 3.     Vehicle constructor runs completely: regNo = "TN09EV0001"             [1] printed first
 4.   Car constructor continues after super(...)                              [2]
 5. ElectricCar constructor continues: batteryKwh = 40                        [3] printed last
 Rule: the parent part of an object is always built before the child part – top of the hierarchy first.
```

The trace in the output confirms the order Vehicle → Car → ElectricCar.

6.5 Alternative Concept and Complexity Analysis – composition (has-a `VehicleType`)

Solution. A single concrete class VehicleC holds regNo and a reference to a VehicleType object that carries name and rate: new VehicleC("TN02B2", new VehicleType("Car", 30)). calculateFee reads type.rate. Implemented in Task06Demo.java; the output shows Van TN07V7 fee(130) = 135.0 created **without writing any new class**.

Criterion	Inheritance (Car is-a Vehicle)	Composition (Vehicle has-a VehicleType)

Add a new vehicle type that differs only in rate	New subclass (≈ 4 lines) + recompile	New VehicleType **data** object – no new class, can even be loaded from a file
Add a type with different *behaviour* (flat fee, peak pricing, Task 12)	Override calculateFee in the new subclass – natural	The behaviour has to be placed in VehicleType as well (a strategy object) or an if on the type – composition alone does not carry behaviour
Type-specific capabilities (Chargeable)	Interface on the subclass; instanceof Chargeable works	Capability must be modelled as a field/flag
Compile-time checking	A Car is a distinct type	Every vehicle has the same Java type; mistakes are caught only at run time
Time to compute a fee	O(1) (virtual call); measured 2.14 ns	O(1) (field read); measured 0.53 ns
Space per vehicle	1 object: header + regNo reference	1 object: header + regNo + one extra reference to a shared VehicleType (≈ 4 bytes more; the types themselves are shared)

Analysis. Both are O(1) in time. The composition version was faster in the micro-benchmark because the JIT can inline a field read while the virtual call with two receiver types needs a dispatch; the absolute numbers are nanoseconds. Composition is the better fit when types differ only in **data**; inheritance when they differ in **behaviour or capabilities**. The campus has both kinds of difference (rates, plus EV charging and later peak pricing), so inheritance is retained with the option (Task 12) of moving rates into data if many rate-only types appear.

```
ElectricCar inherits getRegNo() from Vehicle: TN09EV0001
fee(130 min): bike = 30.0, ev = 90.0


=== Users ===
Student Asha fee-exempt on 2026-11-07 = true
Staff Dr. Rao fee-exempt on 2026-11-07 = false
Visitor Guest fee-exempt on 2026-11-07 = false
```

```
=== Alternative: composition ===
Car TN02B2 fee(130) = 90.0
TwoWheeler TN01A1 fee(130) = 30.0
Van TN07V7 fee(130) = 135.0   (new type = new DATA, no new class)


Fee computation per vehicle (ns): inheritance = 2.14, composition = 0.53 (sink=1)
Space per vehicle: inheritance = 1 object (regNo ref); composition = 1 object (regNo ref + type ref),
VehicleType objects are shared.
Both: fee computation O(1).
```

Task 7 – Override and Specialise Behaviour (4 marks)

7.1 Which methods are overridden, and why

Method	Overridden in	Why the subclass version is needed
ratePerHour() (abstract in Vehicle)	TwoWheeler (10), Car (30)	The ONLY thing that differs between vehicle kinds in the base system is the rate. calculateFee() is written once in Vehicle and calls ratePerHour(), which each subclass answers (template-method pattern). This is the "fee method" that every subclass specialises.
calculateFee(int)	FlatFeeBike (Task07Demo), EBike and DeliveryVan (Task 12)	Behaviour that differs by more than the rate: flat fee, peak pricing. It is deliberately **not final**.
isFeeExempt() / User.payable()	Student, Staff, Visitor	A pass holder pays 0, a visitor the full fee
toString()	Vehicle, ElectricCar (adds (EV) using super.toString())	Readable text instead of ClassName@hash
equals() / hashCode()	Vehicle	Two vehicles with the same registration number are the same vehicle

7.2 `@Override` and the rules of overriding

- **Same signature** (name and parameter types) – otherwise it is overloading, not overriding.
- **Return type** the same or a covariant (sub)type.
- **Access** must not be reduced: a public method cannot be overridden as protected.
- **Checked exceptions** may not be widened. static, private and final methods cannot be overridden.
- @Override makes the compiler check all of this. Compiler evidence (Task07/negative, output in compile_errors.txt):

```
### javac negative/BadFinal.java
negative/BadFinal.java:2: error: cannot assign a value to final variable x
negative/BadFinal.java:3: error: id() in B cannot override id() in A
  overridden method is final
### javac negative/BadOverrideAccess.java
```

```
negative/BadOverrideAccess.java:3: error: fee(int) in C cannot override fee(int) in P
  attempting to assign weaker access privileges; was public
### javac negative/BadOverrideTypo.java
negative/BadOverrideTypo.java:4: error: method does not override or implement a method from a supertype
```

7.3 Use of `final`

Where	Example	Reason
Variable (field)	private final String regNo in Vehicle; final fields in ParkingTicket	Identity and ticket data must not change after creation
Constant	static final int DAILY_CAP_HOURS = 8	Business rule; one value for all objects; cannot be reassigned (compile error shown above)
Method	public final String getRegNo()	No subclass may alter how the identity is read, which equals() relies on
Class	public final class ParkingTicket, DayPass	Immutable value classes must not be subclassed with mutable behaviour

calculateFee is *not* final because the changed requirements in Task 12 need subclasses to override it.

7.4 `toString()`, `equals()`, `hashCode()`

```
@Override public String toString() { return getClass().getSimpleName() + "[" + regNo + ", Rs " + ratePerHour() + "/h]"; }
@Override public boolean equals(Object o) { return o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo); }
@Override public int hashCode() { return regNo.hashCode(); }
```

equals compares only the registration number, so new Car("TN02B2").equals(new ElectricCar("TN02B2")) is true and the relation is symmetric. hashCode uses the same field, so equal objects have equal hash codes (the contract); HashSet.contains(new ElectricCar("TN02B2")) finds the Car – duplicate detection works. Without these overrides Object.equals compares memory addresses and the answer would be false.

```
=== toString() ===
TwoWheeler[TN01A1, Rs 10.0/h]
Car[TN02B2, Rs 30.0/h]
ElectricCar[TN03C3, Rs 30.0/h] (EV)
```

```
FlatFeeBike[TN04D4, Rs 15.0/h]

=== calculateFee overridden vs inherited (130 minutes) ===
TN01A1 -> Rs 30.0
TN02B2 -> Rs 90.0
TN03C3 -> Rs 90.0
TN04D4 -> Rs 15.0

=== equals() and hashCode() ===
Car(TN02B2).equals(ElectricCar(TN02B2)) = true
symmetric: ElectricCar.equals(Car)      = true
Car(TN02B2).equals(Car(TN99Z9))         = false
hashCodes equal for equal objects       = true
HashSet contains ElectricCar(TN02B2)?     true  -> duplicate registration detected
without equals/hashCode the default Object versions compare memory addresses and the answer would be false.

=== User.payable() overridden ===
```

7.5 Alternative Concept and Complexity Analysis – one `calculateFee()` with a `switch` on a type field

Solution. class SwitchVehicle { final int type; double calculateFee(int minutes) { switch (type) { case TWO: rate = 10; break; case CAR: rate = 30; break; case EV: rate = 30; break; default: throw … } … } } (Task07Demo).

Criterion	Overriding	`switch` on a type field
Time per fee for k types	O(1): one virtual dispatch regardless of k; measured 10.08 ns (3 types, megamorphic call site)	O(1) with a compiled tableswitch (O(k) if it is an if–else chain or a string switch); measured 1.07 ns
Classes that must change to add a type	**0 existing** (1 new class)	**≥ 1 existing**: the constants and the switch in the one big class – and every other method that switches on the type
Risk	Compiler forces new subclasses to implement ratePerHour()	A forgotten case is found only at run time (default branch)

Analysis. Honest reading of the measurement: for three types the compiled switch is faster than the virtual call (the call site sees three receiver classes, so the JIT cannot inline it). The difference is a few nanoseconds per fee and invisible next to everything else the gate does. The real cost of the switch design is **change**: the class that contains it is edited and re-tested for every new vehicle type (violates the open/closed principle), whereas overriding adds one class and leaves every tested class untouched. Overriding is therefore retained.

Task 8 – Apply Polymorphism (4 marks)

8.1 Compile-time polymorphism (overloading)

Vehicle has two overloads (Task08/src/poly/PolymorphismDemo.java): calculateFee(int minutes) and calculateFee(LocalTime in, LocalTime out) (an overnight stay wraps past midnight). The **compiler** chooses between them from the number and static types of the arguments, before the program runs: one int → first; two LocalTime values → second. calculateFee((byte) 5) selects the int version by widening byte → int. The second overload ends by calling the first, so the fee rules exist in one place.

8.2 Upcasting: one array, many vehicle types

```
Vehicle[] lot = { new TwoWheeler("TN01A1"), new Car("TN02B2"), new ElectricCar("TN03C3")
};   // upcasting
```

Each element is a Vehicle reference (declared type) to an object of a subclass (actual type). Upcasting is implicit and always safe. Through a Vehicle reference only Vehicle members can be called – that is why v instanceof Chargeable c is needed to reach chargingFee().

8.3 Runtime polymorphism – trace of one call through dynamic method dispatch

```
Statement : double fee = v.calculateFee(130);        // v is the 3rd array element
Declared type of v ..... Vehicle          (the compiler checks that Vehicle has calculateFee(int))
Actual type of the object . ElectricCar      (known only at run time)
Step 1  calculateFee(int) is NOT overridden in ElectricCar or Car → Vehicle.calculateFee executes
Step 2  inside it, this.ratePerHour() is resolved from the ACTUAL type: ElectricCar has no override,
        so the nearest one up the chain is used → Car.ratePerHour() executes → returns 30
Step 3  hours = (130 + 59) / 60 = 3  →  3 × 30 = 90.0
Same statement for the TwoWheeler element: Step 2 runs TwoWheeler.ratePerHour() → 10 → Rs
30.0
```

```
=== 3. Dynamic method dispatch trace ===
call: v.calculateFee(130)   declared type = Vehicle, actual type = TwoWheeler
  calculateFee(int) is not overridden -> Vehicle.calculateFee runs, and inside it:
    this.ratePerHour() is resolved at RUN TIME from the actual object:
    [executing] TwoWheeler.ratePerHour()
  result = Rs 30.0
call: v.calculateFee(130)   declared type = Vehicle, actual type = Car
  calculateFee(int) is not overridden -> Vehicle.calculateFee runs, and inside it:
    this.ratePerHour() is resolved at RUN TIME from the actual object:
```

```
    [executing] Car.ratePerHour()
  result = Rs 90.0
call: v.calculateFee(130)   declared type = Vehicle, actual type = ElectricCar
  calculateFee(int) is not overridden -> Vehicle.calculateFee runs, and inside it:
    this.ratePerHour() is resolved at RUN TIME from the actual object:
    [executing] Car.ratePerHour()
```

8.4 Why this avoids long if–else chains

The calling code (for (Vehicle v : parked) fee = v.calculateFee(130)) never asks "what type are you?". Adding a vehicle type does not touch the loop, the billing code or any if. The instanceof Chargeable test that remains tests a **capability**, not a type, and is needed only because charging is optional.

8.5 Complete the Code – FeeDemo (Tasks 6–9)

The program was copied to Task08/src/FeeDemo.java; the three TODOs were replaced and nothing else changed:

```
// TODO 1 (Vehicle.calculateFee)
int hours = (minutes + 59) / 60;          // ceiling(minutes / 60) without any double
return hours * ratePerHour();
// TODO 2 (Car)
Car(String regNo) { super(regNo); }
@Override protected double ratePerHour() { return 30; }
// TODO 3 (ElectricCar)
@Override public double chargingFee(int kWh) { return kWh * 12; }
```

Expected output	Actual output
TwoWheeler TN01A1 -> Rs 30.0 Car TN02B2 -> Rs 90.0 ElectricCar TN03C3 -> Rs 210.0 true	TwoWheeler TN01A1 -> Rs 30.0 Car TN02B2 -> Rs 90.0 ElectricCar TN03C3 -> Rs 210.0 true

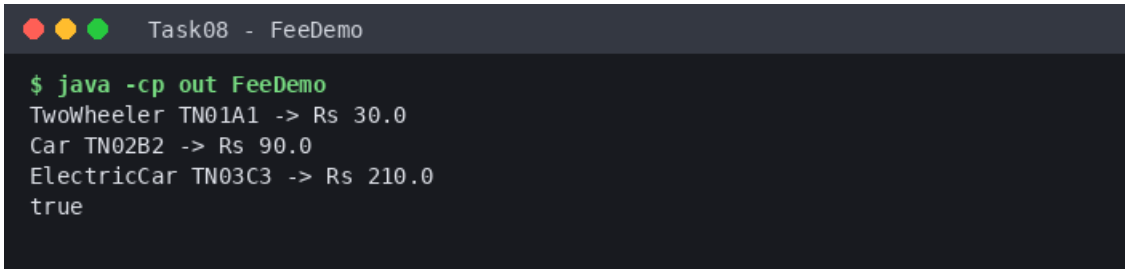


Figure 8.1 – Output of FeeDemo

Check of the numbers: 130 minutes = 3 started hours. Two-wheeler 3 × 10 = 30; car 3 × 30 = 90; EV 3 × 30 + 10 kWh × 12 = 90 + 120 = 210.

Why the last line prints `true`, and is it correct?

Vehicle.equals is o instanceof Vehicle && regNo.equals(((Vehicle) o).regNo). new Car("TN02B2") and new ElectricCar("TN02B2") are both Vehicle objects (an ElectricCar is-a Car is-a Vehicle) with the **same registration number**, so the test passes. **This is the correct behaviour for the system:** a registration number identifies one physical vehicle, which cannot be a Car and an ElectricCar at once, so the same number appearing twice is a duplicate entry that must be refused (Task 11, test 22) – even if the operator picked a different category by mistake. hashCode uses the same field, so hash-based collections agree. The trade-off is that equals is deliberately not "same class"; if two different vehicle types could ever legitimately share a number, getClass() comparison would be needed.

8.6 Alternative Concept and Complexity Analysis – `instanceof` and casts

```
static double fee(Vehicle v, int minutes) {
    int hours = (minutes + 59) / 60;
    if (v instanceof TwoWheeler) return hours * 10;
    else if (v instanceof ElectricCar) return hours * 30;   // subclass must be tested BEFORE its parent Car
    else if (v instanceof Car) return hours * 30;
    throw new IllegalArgumentException("unknown type");
}
```

Criterion	Dynamic dispatch	instanceof chain
Time per call, k types	O(1) – one virtual-table lookup; measured 2.94 ns	O(k) worst case – up to k type tests; measured 0.63 ns for k = 3
Code size	Loop of 3 lines; each type's rule lives in its class	One method that grows by a branch for every type
Maintenance	New type = new class, no edits	Every instanceof chain in the program must be found and edited; order of tests matters (the ElectricCar before Car bug); a forgotten case fails only at run time

Analysis. For k = 3 the chain was faster in the measurement because the type tests are very cheap and predictable, while dispatch at a call site with three receiver types is not inlined; but the chain is O(k)

and for k = 30 types it would need up to 30 tests per call, whereas dispatch stays O(1). Even more important is that the chain duplicates the rates that already live in the classes and has to be edited for every new type. Dynamic dispatch is retained.

Task 9 – Introduce Abstraction (4 marks)

9.1 What is abstract and what is an interface

Type	Kind	Abstract / default members	Why
Vehicle	**abstract class**	abstract ratePerHour(), abstract category(); concrete calculateFee(), getRegNo()	A vehicle IS-A thing with **state** (regNo), a **constructor** that validates it, and an algorithm shared by all subtypes. Only a class can hold these. It is abstract because "a vehicle" with no category has no rate and must never be instantiated.
Chargeable	**interface**	chargingFee(kWh) (default), constant RATE_PER_KWH, static isValidKwh	Charging is an optional **capability**: it cuts across the hierarchy (a future electric two-wheeler could be Chargeable too) and needs no state of its own.
Payable	**interface**	amountDue(fee), payerName()	"Someone who owes money" – implemented by visitors and pass holders without forcing them into one class tree.
User	abstract class	abstract isFeeExempt(), role()	Shares name and its validation

9.2 Why an abstract class in one place and an interface in another

- Use an **abstract class** when subtypes share *state and constructor logic* and form a true is-a family (Vehicle). A class can extend only **one** class, so this slot is spent carefully.
- Use an **interface** for a *capability or role* that unrelated classes may share, and when a class needs **several** of them. Interfaces carry no instance state.

9.3 Multiple behaviours through interfaces

ElectricCar extends Car implements Chargeable, Comparable<ElectricCar> – one parent class and two interfaces: it is a Vehicle, it can be charged, and it can be sorted (the demo sorts by registration number). An ElectricCar reference can be used wherever a Vehicle, a Chargeable or a Comparable is expected.

9.4 Default and static methods in an interface

- **Default method** chargingFeeWithGst(kWh) returns chargingFee(kWh) × 1.18: every Chargeable gets it free, and adding it to the interface later breaks no existing implementing class (output: 141.6 for 10 kWh).
- **Static method** Chargeable.isValidKwh(kWh) is a utility that belongs to the concept (output: isValidKwh(250) = false); it is called on the interface, not on an object.

```
=== Abstract class: template method + abstract hook ===
Car fee(130) = 90.0


=== Interfaces: multiple capabilities ===
ElectricCar is a Vehicle: true, Chargeable: true, Comparable: true
chargingFee(10 kWh)         = 120.0
chargingFeeWithGst(10 kWh)  = 141.6   (default method, written once in the interface)
Chargeable.isValidKwh(250)  = false   (static interface method)
Guest owes Rs 90.0
Asha owes Rs 0.0
sorted via Comparable: TN01, TN09


=== Alternative design (Vehicle = interface, Chargeable = abstract class) ===
CarI fee(130) = 90.0, ElectricCarI fee(130) = 90.0, charge(10) = 120.0
```

9.5 Alternative Concept and Complexity Analysis – Vehicle as interface, Chargeable as abstract class

Solution (in Task09Demo.java): interface VehicleI { String getRegNo(); double ratePerHour(); default double calculateFee(int m) {…} } and abstract class ChargeableA { abstract double chargingFee(int kWh); }. CarI implements VehicleI and ElectricCarI extends ChargeableA implements VehicleI. It compiles and gives the same fees (output), but:

Needed feature	Possible?	Evidence
Instance state (regNo) in VehicleI	**No** – a field in an interface is a public static final constant; every implementing class must declare and validate its own regNo	cannot assign a value to static final variable regNo

Constructor in the interface (super(regNo))	**No** – interfaces have no constructors; the validation code is repeated in every class	VehicleI is abstract; cannot be instantiated
ElectricCar is-a Car **and** Chargeable (abstract class)	**No** – only one class can be extended; ElectricCarI is no longer a CarI, and Chargeable as a class would block every class that already has a parent	'{' expected after extends CarX, ChargeableA
protected / package-private members	**No** – interface methods are implicitly public	–

```
$ for f in negative/*.java; do echo "### javac $f"; javac -d /tmp/neg $f 2>&1; done
### javac negative/NoInterfaceInstance.java
negative/NoInterfaceInstance.java:3: error: VehicleI is abstract; cannot be instantiated
class UseIt { void f() { VehicleI v = new VehicleI(); } }
                                     ^
1 error
### javac negative/NoInterfaceState.java
negative/NoInterfaceState.java:3: error: cannot assign a value to static final variable regNo
class Assign implements VehicleI { void set() { regNo = "Y"; } }
                                                ^
1 error
### javac negative/NoMultipleClassInheritance.java
negative/NoMultipleClassInheritance.java:4: error: '{' expected
class ElectricCarX extends CarX, ChargeableA { }
                                 ^
1 error
```

Time and space. A call through an abstract-class method and through an interface default method are both O(1); measured 0.63 ns versus 0.73 ns – no meaningful difference. Per-object space is the same (the regNo reference is stored in each object either way), but the interface design repeats the field, getter and validation in every implementing class, so the *code* grows with every vehicle type. **Conclusion:** the original split (state-holding abstract class for the is-a family, interfaces for capabilities) is retained; the alternative loses state sharing, constructor sharing and the ability to combine a parent class with a capability class.

Task 10 – Enforce Encapsulation through Packages and Access Modifiers (3 marks)

10.1 Package structure

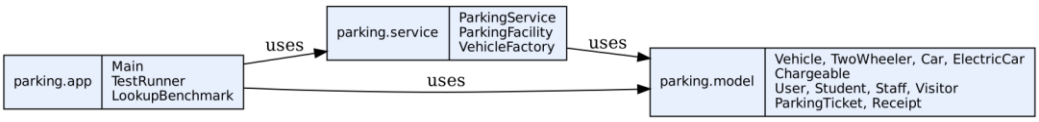


Figure 10.1 – Package structure of the final system (Task 11/12)

Package	Contents	Reason
parking.model	Vehicle, TwoWheeler, Car, ElectricCar, Chargeable, User, Student, Staff, Visitor, ParkingTicket, Receipt (+ Pass, DayPass, EBike, DeliveryVan in Task 12)	Data and the rules that belong to the data; depends on nothing else
parking.service	ParkingFacility, ParkingService, VehicleFactory	Use cases that coordinate model objects; depends on model only
parking.app	Main, TestRunner, benchmarks	Entry points; depends on both. Dependencies point one way (app → service → model), so the model can be tested without the rest.

10.2 Access modifiers (Task 10 demonstration, `ParkingRecord`)

Member	Modifier	Why
Fields regNo, floor, slot, durationMinutes, fee	private	State can change only through validating methods
Getters, validating setters, bill()	public	The API other packages may call
ratePerHourOrDefault()	protected	A hook for subclasses (EvParkingRecord)
isValidRegNo()	default (package-private)	Helper visible inside parking.model only
MAX_MINUTES	private static final	Internal rule
MAX_FLOORS	public static final	Constant other classes may read

No setFee()	–	A fee can only be produced by bill(), never typed in

In the final system the same idea is applied: Vehicle.regNo is private and read through a final getter; ParkingTicket has only private final fields and getters; ParkingFacility.slots is private, so no outside code can put two vehicles into one slot.

Compiler evidence that the modifiers are enforced from another package (Task10/negative/AccessViolations.java):

```
negative/AccessViolations.java:5: error: durationMinutes has private access in ParkingRecord
negative/AccessViolations.java:6: error: isValidRegNo(String) is not public in ParkingRecord; cannot be accessed from outside package
negative/AccessViolations.java:7: error: ratePerHourOrDefault() has protected access in ParkingRecord
```

10.3 Validating getters and setters

```
public void setDurationMinutes(int minutes) {
   if (minutes < 0) throw new IllegalArgumentException("Duration cannot be negative: " + minutes);
   if (minutes > MAX_MINUTES) throw new IllegalArgumentException("Duration exceeds 7 days: " + minutes);
   this.durationMinutes = minutes;
}
public void setPosition(int floor, int slot) {          // floor and slot are validated TOGETHER
   if (floor < 0 || floor >= MAX_FLOORS) throw new IllegalArgumentException("Invalid floor: " + floor);
   if (slot < 0) throw new IllegalArgumentException("Invalid slot: " + slot);
   this.floor = floor; this.slot = slot;
}
```

```
setDurationMinutes(130)   -> accepted
setPosition(9, 3)         -> REJECTED: Invalid floor: 9
setPosition(2, 14)        -> accepted
setRegNo("hello")         -> REJECTED: Invalid registration number: hello
state after the valid calls: ParkingRecord[TN01AB1111, F2-S14, 130 min, Rs 90.0]


=== Service in another package uses only the public API ===
```

10.4 Wrapper classes and autoboxing

- Map<String, Integer> stores Integer objects, so put(reg, 45) **autoboxes** int → Integer and int m = map.get(reg) **auto-unboxes**.
- **Risk 1 – == on wrappers:** Integer a = 1000, b = 1000; a == b is false (two objects); small values −128..127 come from a cache so 127 == 127 is true. Always use equals() or intValue() (output of the demo).
- **Risk 2 – unboxing null:** int x = map.get("missing") throws NullPointerException. A lookup of a vehicle that is not inside must be checked for null first.
- **Cost:** an Integer is a ~16-byte object; an int is 4 bytes and needs no allocation.

```
ParkingRecord[TN02CD2222, F1-S4, 61 min, Rs 60.0]
ParkingRecord[TN03EV3333, F-1-S-1, 130 min, Rs 90.0]


=== Wrapper classes and autoboxing ===
Integer 127  == 127  : true   (small values come from the Integer cache)
Integer 1000 == 1000 : false  (two different objects -> == compares references!)
Integer 1000 equals  : true   (always use equals() or intValue() for wrapper values)
```

10.5 Alternative Concept and Complexity Analysis – public fields, validation by the callers

Solution. PublicRecord has public String regNo; public int durationMinutes; …. Every piece of calling code (entry screen, exit screen, import tool, report fixer) must check if (minutes < 0) before assigning.

What goes wrong. The demo assigns regNo = "hello", durationMinutes = −500, floor = 99, fee = −1: all accepted – the object is silently invalid and the bad value is discovered much later (a negative fee in the day's total). One forgotten check anywhere corrupts the data. The class can no longer change its internal representation (e.g. minutes → Duration) without breaking every caller, because callers touch the field directly.

Criterion	**Private fields + validating setters**	**Public fields + caller validation**
Time cost of validation per update	O(1) – two comparisons	O(1) – the same comparisons

Places where the same check is written	**1** (the setter)	**n** – one for every piece of code that assigns the field (4 in the example above)
Invalid state possible?	No	Yes – any caller can forget
Changing the representation later	Only the class changes	Every caller changes

Conclusion. The time cost is identical; what differs is the number of places that must be right (1 versus n) and whether an invalid object can exist at all. Encapsulation is retained.

Task 11 – Implement, Test and Defend the Design (5 marks)

11.1 The implementation

The complete system is in Task11/src (14 classes in 3 packages; UML in Figure 1.1 and 10.1). Main reads a day's events from input/commands.txt; TestRunner is a hand-written test harness (no external library) that prints a table of expected versus actual results. Compile and run:

```
cd Task11
javac -d out $(find src -name "*.java")
java -cp out parking.app.TestRunner
java -cp out parking.app.Main < input/commands.txt
java -cp out parking.app.LookupBenchmark
```

Key design points: Vehicle.calculateFee implements the slabs of Task 3 once; subclasses supply the rate; ParkingService.enter checks duplicates and capacity **before** a ticket is created; ParkingService.exit computes the fee from the ticket, releases the slot and removes the ticket (setting the freed array cell to null); ParkingFacility uses the jagged array of Task 2.

```
package parking.service;

import parking.model.*;
import java.time.Duration;
import java.time.LocalDateTime;

/** Use cases: enter and exit. Active tickets are kept in a plain array (the design evaluated in Task 11).
*/
public class ParkingService {
    private final ParkingFacility facility;
    private final ParkingTicket[] active;              // active[0..count-1]
    private int count = 0;
    private double collectedToday = 0;

    public ParkingService(ParkingFacility facility) {
        this.facility = facility;
        this.active = new ParkingTicket[facility.totalSlots()];
    }

    private int indexOf(String regNo) {                // linear search: O(n)
```

```
        for (int i = 0; i < count; i++) if (active[i].getVehicle().getRegNo().equals(regNo)) return i;
        return -1;
    }

    public ParkingTicket enter(Vehicle v, User u, LocalDateTime when) {
        if (indexOf(v.getRegNo()) >= 0) throw new IllegalStateException("Duplicate registration number: " + v.getRegNo());
        int[] pos = facility.allocate(v.getRegNo());
        if (pos == null) throw new IllegalStateException("Parking facility is full");
        ParkingTicket t = new ParkingTicket(v, u, pos[0], pos[1], when);
        active[count++] = t;
        return t;
    }

    public Receipt exit(String regNo, LocalDateTime when, int kWh) {
        int i = indexOf(regNo);
        if (i < 0) throw new IllegalStateException("No active ticket for " + regNo);
        ParkingTicket t = active[i];
        if (when.isBefore(t.getEntryTime())) throw new IllegalArgumentException("Exit time is before entry time");
        int minutes = (int) Duration.between(t.getEntryTime(), when).toMinutes();
        Vehicle v = t.getVehicle();
        double parking = v.calculateFee(minutes);
        boolean exempt = t.getUser().isFeeExempt(t.getEntryTime().toLocalDate());
        double charging = 0;
        if (kWh > 0) {
            if (!(v instanceof Chargeable)) throw new IllegalArgumentException(v + " cannot be charged");
            charging = ((Chargeable) v).chargingFee(kWh);
        }
        facility.release(t.getFloor(), t.getSlot());
        active[i] = active[--count];                    // remove by moving the last ticket into the hole
        active[count] = null;                           // drop the reference -> the ticket can be garbage collected
        Receipt r = new Receipt(t.getTicketNo(), regNo, minutes, parking, exempt, charging);
        collectedToday += r.totalPayable();
        return r;
    }
```

```
    public int activeCount() { return count; }
    public double getCollectedToday() { return collectedToday; }
    public ParkingFacility getFacility() { return facility; }
}
```

11.2 Test table (expected versus actual)

The categories requested in the assignment are covered: normal operation, boundary cases (last free slot, zero-minute parking, every slab boundary), invalid cases (unknown type, duplicate registration, invalid slot) and a hand-calculated day. **Result: 35 / 35 tests passed.**

#	Category	Test	Expected	Actual	Result
1	Normal	TwoWheeler enters -> first slot	[0, 0]	[0, 0]	PASS
2	Normal	Car enters -> second slot	[0, 1]	[0, 1]	PASS
3	Normal	Ticket numbers are consecutive	true	true	PASS
4	Normal	TwoWheeler 90 min -> Rs 20	20.0	20.0	PASS
5	Normal	Slot freed -> next car gets [0, 0]	[0, 0]	[0, 0]	PASS
6	Normal	Student with valid pass parks free	0.0	0.0	PASS
7	Normal	Staff with EXPIRED pass pays 4 h x 30	120.0	120.0	PASS
8	Normal	EV 121 min + 10 kWh = 90 + 120	210.0	210.0	PASS
9	Normal	Pass holder EV still pays charging	120.0	120.0	PASS
10	Boundary	Last free slot is allocated	[1, 0]	[1, 0]	PASS
11	Boundary	Entry when full is refused	IllegalStateException: Parking facility is full	IllegalStateException: Parking facility is full	PASS
12	Boundary	Zero-minute stay costs Rs 0	0.0	0.0	PASS

13	Boundary	1 minute = 1 started hour	30.0	30.0	PASS
14	Boundary	60 minutes (slab edge) = 1 h	30.0	30.0	PASS
15	Boundary	61 minutes (just over) = 2 h	60.0	60.0	PASS
16	Boundary	480 min = 8 h = daily cap reached	240.0	240.0	PASS
17	Boundary	720 min limited by daily cap	240.0	240.0	PASS
18	Boundary	1440 min (exactly one day) = cap	240.0	240.0	PASS
19	Boundary	1441 min = cap + 1 started hour	270.0	270.0	PASS
20	Boundary	Same-minute entry and exit	0.0	0.0	PASS
21	Invalid	Unknown vehicle type 'BUS'	IllegalArgumentException: Unknown vehicle type: BUS	IllegalArgumentException: Unknown vehicle type: BUS	PASS
22	Invalid	Duplicate registration number	IllegalStateException: Duplicate registration number: TN02B2	IllegalStateException: Duplicate registration number: TN02B2	PASS
23	Invalid	Release invalid floor 5	false	false	PASS
24	Invalid	Release negative slot	false	false	PASS
25	Invalid	Release an already-free slot	false	false	PASS
26	Invalid	Negative duration	IllegalArgumentException: Negative duration: -5	IllegalArgumentException: Negative duration: -5	PASS
27	Invalid	Exit of a vehicle that never entered	IllegalStateException: No active ticket for GHOST1	IllegalStateException: No active ticket for GHOST1	PASS
28	Invalid	Exit before entry	IllegalArgumentException: Exit time is before entry time	IllegalArgumentException: Exit time is before entry time	PASS
29	Invalid	Blank registration number	IllegalArgumentException: Invalid registration number:	IllegalArgumentException: Invalid registration number:	PASS

30	Invalid	Charging a non-EV	IllegalArgumentException: Car[CAR5] cannot be charged	IllegalArgumentException: Car[CAR5] cannot be charged	PASS
31	Object	equals: Car vs ElectricCar same regNo	true	true	PASS
32	Object	hashCode equal for equal vehicles	true	true	PASS
33	Day	Hand total 20+0+0+30+210+240	500.0	500.0	PASS
34	Day	All six vehicles have left	0	0	PASS
35	Day	All slots free again (10 of 10)	10	10	PASS

A defect found by the first test run (honest record). The first run reported 33 / 35: tests 10 and 11 failed with Invalid registration number: B1. The cause was in the **test data**, not the program: the registration numbers B1/B2 are shorter than the 4-character minimum that Vehicle enforces (rule: 4–12 letters/digits). The program was right to reject them; the tests were changed to BB11/BB22 and rerun: 35 / 35.

11.3 Hand-calculated day versus program output

Vehicle	Who	Entry → Exit	Minutes	Calculation	Rs
TN01A1 two-wheeler	Visitor	08:00 → 09:30	90	⌈90/60⌉ = 2 h × 10	20
TN02B2 car	Student, monthly pass	08:00 → 17:00	540	pass → parking waived	0
TN03C3 car	Visitor	10:00 → 10:00	0	zero minutes → free	0
TN04D4 car	Visitor	09:00 → 10:00	60	⌈60/60⌉ = 1 h × 30	30
TN05E5 EV	Visitor	09:00 → 11:01	121	⌈121/60⌉ = 3 h × 30 = 90, + 10 kWh × 12 = 120	210
TN06F6 car	Visitor	00:00 → 12:00	720	12 h × 30 = 360, capped at 8 h × 30	240

Total				20 + 0 + 0 + 30 + 210 + 240	**500**

```
$ java -cp out parking.app.Main < input/commands.txt
Facility created: 3 floors, 10 slots
ENTER TwoWheeler[TN01A1] (Visitor) -> Ticket#1000[TN01A1 F0-S0 2026-11-07T08:00]
ENTER Car[TN02B2] (Student, monthly pass) -> Ticket#1001[TN02B2 F0-S1 2026-11-07T08:00]
ENTER Car[TN03C3] (Visitor) -> Ticket#1002[TN03C3 F1-S0 2026-11-07T10:00]
ENTER Car[TN04D4] (Visitor) -> Ticket#1003[TN04D4 F1-S1 2026-11-07T09:00]
ENTER ElectricCar[TN05E5] (Visitor) -> Ticket#1004[TN05E5 F1-S2 2026-11-07T09:00]
ENTER Car[TN06F6] (Visitor) -> Ticket#1005[TN06F6 F2-S0 2026-11-07T00:00]
line 10 ERROR (IllegalArgumentException): Unknown vehicle type: BUS
line 11 ERROR (IllegalStateException): Duplicate registration number: TN02B2
line 12 ERROR (IllegalStateException): No active ticket for GHOST1
EXIT  TN01A1 after 90 min: parking Rs 20.0, charging Rs 0.0 -> PAY Rs 20.0
EXIT  TN02B2 after 540 min: parking Rs 240.0 (waived: pass), charging Rs 0.0 -> PAY Rs 0.0
EXIT  TN03C3 after 0 min: parking Rs 0.0, charging Rs 0.0 -> PAY Rs 0.0
EXIT  TN04D4 after 60 min: parking Rs 30.0, charging Rs 0.0 -> PAY Rs 30.0
EXIT  TN05E5 after 121 min: parking Rs 90.0, charging Rs 120.0 -> PAY Rs 210.0
EXIT  TN06F6 after 720 min: parking Rs 240.0, charging Rs 0.0 -> PAY Rs 240.0
SUMMARY: vehicles inside = 0, free slots = 10, collection today = Rs 500.0
```

The program prints collection today = Rs 500.0, equal to the hand total. The same input also exercises three invalid commands (unknown type BUS, duplicate TN02B2, exit of GHOST1): each is reported on its own line and the program continues.

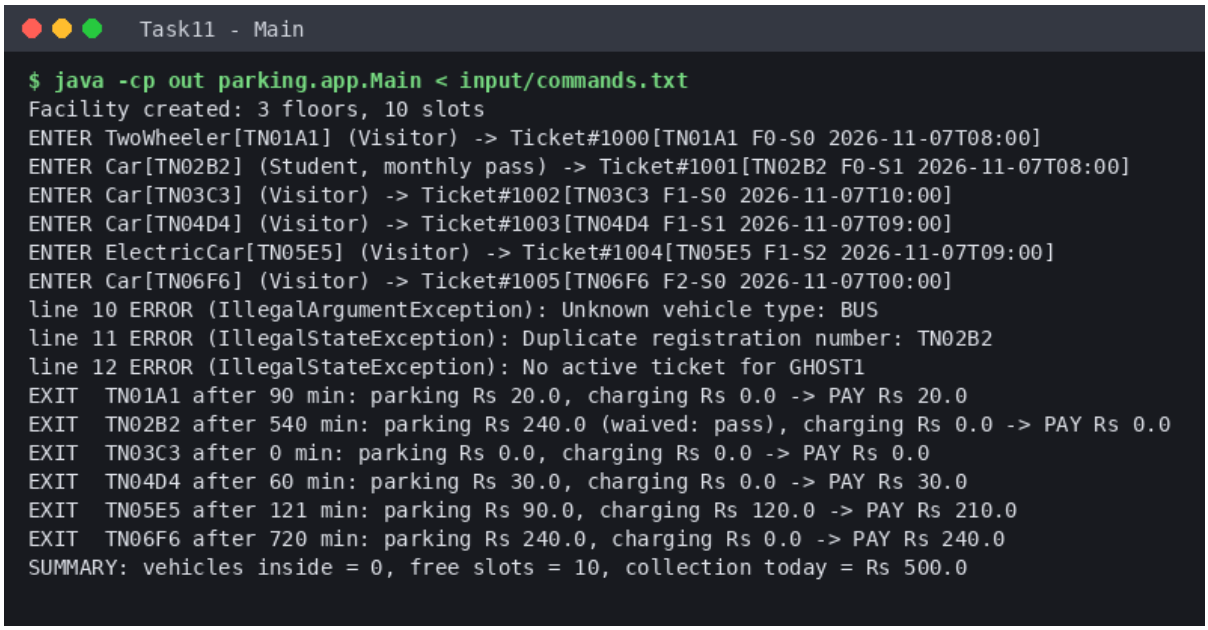


Figure 11.1 – Output of Main for the day scenario

11.4 Evaluation of the design (evidence-based)

Principle	How the implementation follows it	Evidence
Encapsulation	All fields private; Vehicle.regNo final; ParkingTicket and Receipt immutable; slots reachable only through allocate/release	No test can put two vehicles in one slot; negative-compile examples in Task 10
Inheritance	Vehicle → TwoWheeler/Car → ElectricCar; User → three subclasses; shared code written once	Fee algorithm exists once; constructor trace Task 6
Polymorphism	ParkingService calls v.calculateFee(...) and isFeeExempt without any type test	Tests 4, 6–9: four types, four fees, no if on type
Abstraction	Vehicle, User abstract; Chargeable capability interface	ParkingService tests instanceof Chargeable, not a class name

Limitation. Active tickets live in a plain array and are found by a linear search, so exit and the duplicate check on enter are O(n). For the specified facility (≤ 500 vehicles) this costs about a microsecond, but it does not scale (next section). Other limitations: money is a double; charging slots are not a separate pool; the running day total is kept only in memory.

11.5 Alternative Concept and Complexity Analysis – ArrayList or HashMap for active tickets

Exit processing with each structure. *Array (current):* loop i = 0 … count-1 comparing active[i].getVehicle().getRegNo(); remove by moving the last element into the hole.

ArrayList<ParkingTicket>: the same loop with list.get(i), remove with list.remove(i) (shifts the tail: O(n)) or the swap trick. *HashMap<String, ParkingTicket>:* ParkingTicket t = map.remove(regNo); if (t == null) throw … – lookup, duplicate check (containsKey) and removal in one expected O(1) step.

Measured (worst case: the last-inserted ticket is searched; best of 7 warmed-up rounds), nanoseconds per lookup, and extra bytes per ticket for the container:

n parked	**Array (ns)**	**ArrayList (ns)**	**HashMap (ns)**	**ArrayList B/ticket**	**HashMap B/ticket**
100	218.1	313.6	2.7	n/a	n/a
1,000	2706.5	1986.1	3.0	n/a	n/a
10,000	47954.2	30093.6	4.6	5	38
100,000	635303.6	927467.8	13.3	4	42

Analysis. Array and ArrayList both scan: time grows in proportion to n (n = 100 → 100,000 is a 1,000× increase, and the scan time rises by roughly that factor), while HashMap stays at a few nanoseconds: Θ(n) versus expected Θ(1). Space is Θ(n) in all three; the array is cheapest (one 4-byte reference per ticket), the ArrayList adds growth slack, and the HashMap adds about 40 bytes per ticket (a Node object plus a table slot) – the price of constant-time lookup. **Recommendation, based on these measurements:** at the stated maximum of 500 vehicles the array's ≈ 1 µs is acceptable and the simplest design is defensible, but the HashMap is the better engineering choice because it also gives the duplicate check in O(1), costs only ≈ 20 KB at 500 tickets, and keeps the exit time constant if the facility grows. This is the single change recommended before deployment.

Task 12 – Re-evaluate and Adapt the Design (8 marks)

Changed requirements: (1) e-bikes with a flat fee, (2) delivery vans at double the car rate between 9 AM and 5 PM, (3) a one-day pass for visitors, (4) five floors with a different number of slots on each floor (20, 35, 50, 75, 100 = 280 slots). **Decision: MODIFY** the original design (not retain unchanged, not redesign).

A. Impact Analysis (2 marks)

What changed	Affected parts of the original design	Effect
Two new vehicle categories	Vehicle hierarchy; VehicleFactory	New subclasses EBike, DeliveryVan – **extension**. The factory's switch must learn two new words – **modification** (fixed with a registry, see C)
Van price depends on the **time of day**	Vehicle.calculateFee(int minutes) receives only a duration	A duration cannot say whether an hour falls between 09:00 and 17:00. The fee method needs the entry/exit time → Vehicle gets a **time-aware overload**; ParkingService calls it – **modification** of two classes
E-bike flat fee	calculateFee and the abstract ratePerHour()	Override of calculateFee in EBike; ratePerHour() is not meaningful for a flat fee (a small design wart, see B)
Visitors can buy a one-day pass	User hierarchy (Visitor.isFeeExempt returns false)	A visitor may now be exempt → new Pass interface + DayPass; Visitor gets an optional pass – **modification** of one class. ParkingService.exit is unchanged here because it already asks user.isFeeExempt(date) (polymorphism paid off)
5 floors, different sizes	ParkingFacility and the slot data structure	**No change** – the jagged array String[][] was chosen for uneven floors in Task 2

Why different slot counts per floor matter for the array design. A rectangular array or the flat-array formula index = floor × slotsPerFloor + slot assumes equal floors. With floors {20, 35, 50, 75, 100}, the formula would place slot 40 of floor 3 at 3 × 20 + 40 = 100 instead of its true position 145

(demonstrated in Task 2) – two vehicles could be mapped to the same index, or a vehicle to a slot of the wrong floor. The jagged array gives every floor its own length, and allocate/release already loop to slots[f].length, so nothing changes. Test evidence: the 106th vehicle lands on [3, 0] and the 280th on [4, 99].

Modified rather than extended: Vehicle (one added method), Visitor (one added field and constructor), ParkingService (one changed line), VehicleFactory (switch replaced by a registry). Everything else was extended or left alone (counts in D).

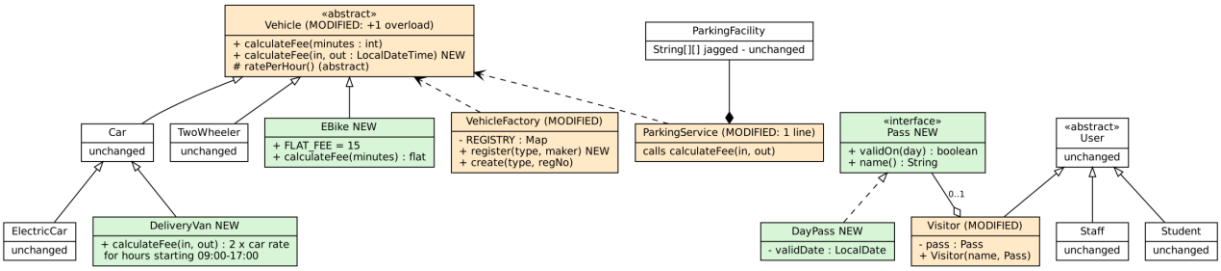


Figure 12.1 – Revised class structure (green = new, orange = modified, white = unchanged)

B. Evaluation of the Original Design (2 marks)

Question	Finding	Evidence
Can new vehicle types be added only by writing new subclasses?	**Yes for behaviour, with one caveat.** Both new categories are new subclasses (EBike, DeliveryVan) and no existing subclass changed. The caveat: the original calculateFee(int) could not express time-of-day pricing, so one base-class method had to be **added** (not changed). The original VehicleFactory was a switch that had to be edited for each new type.	Diff in D; TwoWheeler, Car, ElectricCar byte-identical
Are pricing rules spread across many classes or in one place?	**Spread.** The hourly slab and daily cap are in Vehicle, the rate in each subclass, the flat fee in EBike, the peak rule in DeliveryVan, the exemption in User subclasses. A pricing change therefore means finding the right class. This is the main weakness of the design: a PricingRule abstraction (one class per rule, chosen per vehicle category) would centralise it.	Source review; EBike.ratePerHour() exists only because the abstract base requires it

Can the existing tested classes remain unchanged?	**Largely yes.** 10 of 14 model/service classes are untouched, and the **35 original tests, run unchanged against the new code, still pass**.	Result: 35 / 35 tests passed

C. Adaptation or Redesign (2 marks)

Decision: MODIFY. A full redesign (central PricingRule strategy for every vehicle) was considered and rejected for now: it would have changed all 14 classes and invalidated the tested code to fix a weakness that costs nothing yet, whereas the targeted changes below satisfy every new requirement. The trigger for the redesign is stated in D.

What changed

1. **Vehicle:** added calculateFee(LocalDateTime in, LocalDateTime out), default = calculateFee(minutes between). Existing callers and subclasses are unaffected (an *overload*, not a change).
2. **EBike** (new): overrides calculateFee(int) → flat Rs 15, zero minutes free.
3. **DeliveryVan** (new, extends Car): overrides the time-aware overload → per-hour peak pricing with a per-24-hour cap.
4. **Pass / DayPass** (new): Pass.validOn(date); a DayPass is valid only on its own date. Visitor(name, Pass) is a new constructor; isFeeExempt asks the pass. Student/Staff are unchanged.
5. **VehicleFactory** (modified): switch replaced by a Map<String, Function<String, Vehicle>> registry with a public register(), so future types are added without editing the factory.
6. **ParkingService** (modified, one line): v.calculateFee(t.getEntryTime(), when) instead of v.calculateFee(minutes).
7. **ParkingFacility:** unchanged – the jagged array already supports {20, 35, 50, 75, 100}.

Key code

```
package parking.model;

import java.time.Duration;
import java.time.LocalDateTime;
import java.time.LocalTime;

/**
 * Delivery van: DOUBLE the car rate for any billable hour that STARTS between 09:00 (inclusive) and 17:00 (exclusive).
 * Extends Car so that a future change of the car rate flows through automatically.
```

```
 */
public class DeliveryVan extends Car {
   public static final LocalTime PEAK_START = LocalTime.of(9, 0);
   public static final LocalTime PEAK_END = LocalTime.of(17, 0);
   public static final int PEAK_MULTIPLIER = 2;

   public DeliveryVan(String regNo) { super(regNo); }

   private static boolean isPeak(LocalDateTime t) {
      LocalTime lt = t.toLocalTime();
      return !lt.isBefore(PEAK_START) && lt.isBefore(PEAK_END);
   }

   @Override public double calculateFee(LocalDateTime in, LocalDateTime out) {
      if (out.isBefore(in)) throw new IllegalArgumentException("Exit time is before entry time");
      long minutes = Duration.between(in, out).toMinutes();
      if (minutes == 0) return 0;
      int hours = (int) ((minutes + 59) / 60);
      double cap = DAILY_CAP_HOURS * PEAK_MULTIPLIER * ratePerHour();     // per 24-hour
block
      double total = 0, block = 0;
      int inBlock = 0;
      for (int i = 0; i < hours; i++) {
         LocalDateTime hourStart = in.plusHours(i);
         block += isPeak(hourStart) ? PEAK_MULTIPLIER * ratePerHour() : ratePerHour();
         inBlock++;
         if (inBlock == 24 || i == hours - 1) { total += Math.min(block, cap); block = 0; inBlock = 0; }
      }
      return total;
   }
   @Override public String category() { return "DeliveryVan"; }
}
```

```
package parking.model;
/** E-bike: ONE flat fee per visit, whatever the duration (zero-minute stays are still free). */
public class EBike extends Vehicle {
   public static final double FLAT_FEE = 15;
```

```
    public EBike(String regNo) { super(regNo); }
    @Override protected double ratePerHour() { return FLAT_FEE; }      // required by the abstract base; not used for billing
    @Override public double calculateFee(int minutes) {
        if (minutes < 0) throw new IllegalArgumentException("Negative duration: " + minutes);
        return minutes == 0 ? 0 : FLAT_FEE;
    }
    @Override public String category() { return "EBike"; }
}

package parking.model;
import java.time.LocalDate;
/** One-day pass for visitors: valid only on the day it was bought for. */
public final class DayPass implements Pass {
    private final LocalDate validDate;
    public DayPass(LocalDate validDate) {
        if (validDate == null) throw new IllegalArgumentException("validDate required");
        this.validDate = validDate;
    }
    @Override public boolean validOn(LocalDate day) { return validDate.equals(day); }
    @Override public String name() { return "one-day pass " + validDate; }
}
```

```
package parking.service;

import parking.model.*;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.function.Function;

/** Creates vehicles from gate text. New categories are REGISTERED here - no other class has to change. */
public final class VehicleFactory {
    private static final Map<String, Function<String, Vehicle>> REGISTRY = new LinkedHashMap<>();
    static {
        register("TWO", TwoWheeler::new);   register("TWOWHEELER", TwoWheeler::new);
```

```
        register("CAR", Car::new);
        register("EV", ElectricCar::new);   register("ELECTRICCAR", ElectricCar::new);
        register("EBIKE", EBike::new);
        register("VAN", DeliveryVan::new);
    }
    private VehicleFactory() { }
    public static void register(String type, Function<String, Vehicle> maker) {
REGISTRY.put(type.toUpperCase(), maker); }
    public static Vehicle create(String type, String regNo) {
        Function<String, Vehicle> maker = type == null ? null : REGISTRY.get(type.toUpperCase());
        if (maker == null) throw new IllegalArgumentException("Unknown vehicle type: " + type);
        return maker.apply(regNo);
    }
}
```

Pricing rule for the van (stated explicitly because the text does not say how a stay that crosses 09:00 or 17:00 is priced): the stay is billed in started hours as before; an hour is **peak** (2 × car rate = Rs 60) when it **starts** at or after 09:00 and before 17:00, otherwise Rs 30; each 24-hour block is capped at 8 peak-hours' worth (Rs 480). Example: 08:30–10:30 → hour 1 starts 08:30 (30) + hour 2 starts 09:30 (60) = Rs 90.

D. Evidence-Based Defence (2 marks)

1. Number of existing classes that had to change

```
$ cd .. && diff -rq Task11/src Task12/src | sort
Files Task11/src/parking/app/Main.java and Task12/src/parking/app/Main.java differ
Files Task11/src/parking/model/Vehicle.java and Task12/src/parking/model/Vehicle.java differ
Files Task11/src/parking/model/Visitor.java and Task12/src/parking/model/Visitor.java differ
Files Task11/src/parking/service/ParkingService.java and
Task12/src/parking/service/ParkingService.java differ
Files Task11/src/parking/service/VehicleFactory.java and
Task12/src/parking/service/VehicleFactory.java differ
Only in Task11/src/parking/app: LookupBenchmark.java
Only in Task12/src/parking/app: Task12Tests.java
Only in Task12/src/parking/model: DayPass.java
Only in Task12/src/parking/model: DeliveryVan.java
Only in Task12/src/parking/model: EBike.java
Only in Task12/src/parking/model: Pass.java
```

Group	Classes	Count
Modified (model + service)	Vehicle, Visitor, ParkingService, VehicleFactory	4 of 14 existing (29 %)
Unchanged	TwoWheeler, Car, ElectricCar, User, Student, Staff, Chargeable, ParkingTicket, Receipt, ParkingFacility	10 of 14 (71 %)
New	EBike, DeliveryVan, Pass, DayPass	4
Driver	Main – parses the new DAYPASS token (the model and service did not need this)	1 (app layer)

2. Ease of adding a further vehicle type in the future

A new type needs **one new class** and **one register(...) call**; no existing class is edited. Test evidence (Task12Tests, category *Future*): a TRUCK (Rs 50/hour) was registered at run time with a three-line lambda in the test itself and produced Truck[TN88T8] and a fee of Rs 150 for 130 minutes. In the original design the same change meant editing the factory's switch.

3. Test results for the new categories

Regression: the original 35 tests, same file, new code: **35 / 35 tests passed**. New tests: **31 / 31 passed** (e-bike, van boundaries at 09:00/17:00, overnight and daily-cap stays, day pass today / yesterday / with charging, uneven floors, factory, future type, hand-calculated day = Rs 285).

#	Category	Test	Expected	Actual	Result
1	EBike	0 minutes is free	0.0	0.0	PASS
2	EBike	1 minute = flat Rs 15	15.0	15.0	PASS
3	EBike	10 hours = still Rs 15	15.0	15.0	PASS
4	EBike	5 days = still Rs 15	15.0	15.0	PASS
5	Van	10:00-12:00 = 2 peak h x Rs 60	120.0	120.0	PASS
6	Van	18:00-19:00 = 1 off-peak h x Rs 30	30.0	30.0	PASS
7	Van	08:30-10:30 = 30 + 60 (straddles 09:00)	90.0	90.0	PASS

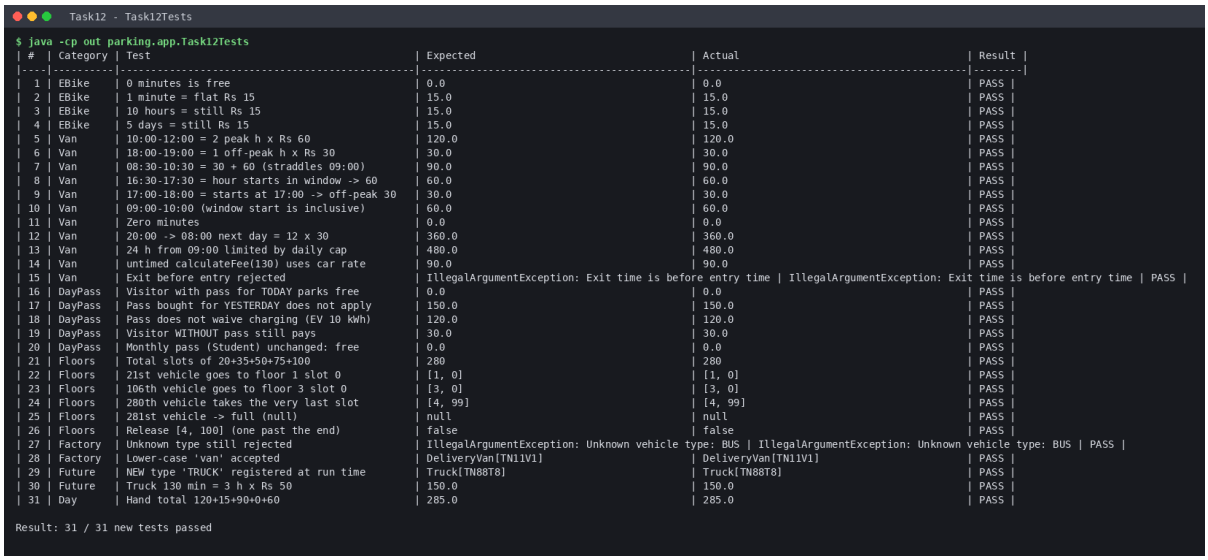
8	Van	16:30-17:30 = hour starts in window -> 60	60.0	60.0	PASS
9	Van	17:00-18:00 = starts at 17:00 -> off-peak 30	30.0	30.0	PASS
10	Van	09:00-10:00 (window start is inclusive)	60.0	60.0	PASS
11	Van	Zero minutes	0.0	0.0	PASS
12	Van	20:00 -> 08:00 next day = 12 x 30	360.0	360.0	PASS
13	Van	24 h from 09:00 limited by daily cap	480.0	480.0	PASS
14	Van	untimed calculateFee(130) uses car rate	90.0	90.0	PASS
15	Van	Exit before entry rejected	IllegalArgumentException: Exit time is before entry time	IllegalArgumentException: Exit time is before entry time	PASS
16	DayPass	Visitor with pass for TODAY parks free	0.0	0.0	PASS
17	DayPass	Pass bought for YESTERDAY does not apply	150.0	150.0	PASS
18	DayPass	Pass does not waive charging (EV 10 kWh)	120.0	120.0	PASS
19	DayPass	Visitor WITHOUT pass still pays	30.0	30.0	PASS
20	DayPass	Monthly pass (Student) unchanged: free	0.0	0.0	PASS
21	Floors	Total slots of 20+35+50+75+100	280	280	PASS

22	Floors	21st vehicle goes to floor 1 slot 0	[1, 0]	[1, 0]	PASS
23	Floors	106th vehicle goes to floor 3 slot 0	[3, 0]	[3, 0]	PASS
24	Floors	280th vehicle takes the very last slot	[4, 99]	[4, 99]	PASS
25	Floors	281st vehicle -> full (null)	null	null	PASS
26	Floors	Release [4, 100] (one past the end)	false	false	PASS
27	Factory	Unknown type still rejected	IllegalArgumentException: Unknown vehicle type: BUS	IllegalArgumentException: Unknown vehicle type: BUS	PASS
28	Factory	Lower-case 'van' accepted	DeliveryVan[TN11V1]	DeliveryVan[TN11V1]	PASS
29	Future	NEW type 'TRUCK' registered at run time	Truck[TN88T8]	Truck[TN88T8]	PASS
30	Future	Truck 130 min = 3 h x Rs 50	150.0	150.0	PASS
31	Day	Hand total 120+15+90+0+60	285.0	285.0	PASS

Hand calculation for the new-categories day (input/commands.txt): van 10:00–12:00 = 2 × 60 = 120; e-bike 150 min = flat 15; van 08:30–10:30 = 30 + 60 = 90; car with a day pass = 0; car 120 min = 2 × 30 = 60. **Total 120 + 15 + 90 + 0 + 60 = Rs 285**, equal to the program's collection today = Rs 285.0.

```
$ java -cp out parking.app.Main < input/commands.txt
Facility created: 5 floors, 280 slots
ENTER DeliveryVan[TN11V1] (Visitor) -> Ticket#1000[TN11V1 F0-S0 2026-11-07T10:00]
ENTER EBike[TN12E1] (Visitor) -> Ticket#1001[TN12E1 F0-S1 2026-11-07T09:00]
ENTER DeliveryVan[TN13V2] (Visitor) -> Ticket#1002[TN13V2 F0-S2 2026-11-07T08:30]
ENTER Car[TN14C1] (Visitor (one-day pass 2026-11-07)) -> Ticket#1003[TN14C1 F0-S3 2026-11-07T09:00]
ENTER Car[TN15C2] (Visitor) -> Ticket#1004[TN15C2 F0-S4 2026-11-07T09:00]
line 9 ERROR (IllegalArgumentException): Unknown vehicle type: BUS
EXIT  TN11V1 after 120 min: parking Rs 120.0, charging Rs 0.0 -> PAY Rs 120.0
```

```
EXIT  TN12E1 after 150 min: parking Rs 15.0, charging Rs 0.0 -> PAY Rs 15.0
EXIT  TN13V2 after 120 min: parking Rs 90.0, charging Rs 0.0 -> PAY Rs 90.0
EXIT  TN14C1 after 300 min: parking Rs 150.0 (waived: pass), charging Rs 0.0 -> PAY Rs 0.0
EXIT  TN15C2 after 120 min: parking Rs 60.0, charging Rs 0.0 -> PAY Rs 60.0
SUMMARY: vehicles inside = 0, free slots = 280, collection today = Rs 285.0
```



```
Task12 - Task12Tests

$ java -cp out parking.app.Task12Tests
| #  | Category | Test                                          | Expected                                  | Actual                                    | Result |
|----|----------|-----------------------------------------------|-------------------------------------------|-------------------------------------------|--------|
|  1 | EBike    | 0 minutes is free                             | 0.0                                       | 0.0                                       | PASS |
|  2 | EBike    | 1 minute = flat Rs 15                         | 15.0                                      | 15.0                                      | PASS |
|  3 | EBike    | 10 hours = still Rs 15                        | 15.0                                      | 15.0                                      | PASS |
|  4 | EBike    | 5 days = still Rs 15                          | 15.0                                      | 15.0                                      | PASS |
|  5 | Van      | 10:00-12:00 = 2 peak h x Rs 60                | 120.0                                     | 120.0                                     | PASS |
|  6 | Van      | 18:00-19:00 = 1 off-peak h x Rs 30            | 30.0                                      | 30.0                                      | PASS |
|  7 | Van      | 08:30-10:30 = 30 + 60 (straddles 09:00)       | 90.0                                      | 90.0                                      | PASS |
|  8 | Van      | 16:30-17:30 = hour starts in window -> 60     | 60.0                                      | 60.0                                      | PASS |
|  9 | Van      | 17:00-18:00 = starts at 17:00 -> off-peak 30  | 30.0                                      | 30.0                                      | PASS |
| 10 | Van      | 09:00-10:00 (window start is inclusive)       | 60.0                                      | 60.0                                      | PASS |
| 11 | Van      | Zero minutes                                  | 0.0                                       | 0.0                                       | PASS |
| 12 | Van      | 20:00 -> 08:00 next day = 12 x 30             | 360.0                                     | 360.0                                     | PASS |
| 13 | Van      | 24 h from 09:00 limited by daily cap          | 480.0                                     | 480.0                                     | PASS |
| 14 | Van      | untimed calculateFee(130) uses car rate       | 90.0                                      | 90.0                                      | PASS |
| 15 | Van      | Exit before entry rejected                    | IllegalArgumentException: Exit time is before entry time | IllegalArgumentException: Exit time is before entry time | PASS |
| 16 | DayPass  | Visitor with pass for TODAY parks free        | 0.0                                       | 0.0                                       | PASS |
| 17 | DayPass  | Pass bought for YESTERDAY does not apply      | 150.0                                     | 150.0                                     | PASS |
| 18 | DayPass  | Pass does not waive charging (EV 10 kWh)      | 120.0                                     | 120.0                                     | PASS |
| 19 | DayPass  | Visitor WITHOUT pass still pays               | 30.0                                      | 30.0                                      | PASS |
| 20 | DayPass  | Monthly pass (Student) unchanged: free        | 0.0                                       | 0.0                                       | PASS |
| 21 | Floors   | Total slots of 20+35+50+75+100                | 280                                       | 280                                       | PASS |
| 22 | Floors   | 21st vehicle goes to floor 1 slot 0           | [1, 0]                                    | [1, 0]                                    | PASS |
| 23 | Floors   | 106th vehicle goes to floor 3 slot 0          | [3, 0]                                    | [3, 0]                                    | PASS |
| 24 | Floors   | 280th vehicle takes the very last slot        | [4, 99]                                   | [4, 99]                                   | PASS |
| 25 | Floors   | 281st vehicle -> full (null)                  | null                                      | null                                      | PASS |
| 26 | Floors   | Release [4, 100] (one past the end)           | false                                     | false                                     | PASS |
| 27 | Factory  | Unknown type still rejected                   | IllegalArgumentException: Unknown vehicle type: BUS | IllegalArgumentException: Unknown vehicle type: BUS | PASS |
| 28 | Factory  | Lower-case 'van' accepted                     | DeliveryVan[TN11V1]                       | DeliveryVan[TN11V1]                       | PASS |
| 29 | Future   | NEW type 'TRUCK' registered at run time       | Truck[TN88T8]                             | Truck[TN88T8]                             | PASS |
| 30 | Future   | Truck 130 min = 3 h x Rs 50                   | 150.0                                     | 150.0                                     | PASS |
| 31 | Day      | Hand total 120+15+90+0+60                     | 285.0                                     | 285.0                                     | PASS |

Result: 31 / 31 new tests passed
```

Figure 12.2 – Output of the new Task 12 tests

4. Trade-offs and honest weaknesses

- **Flat-fee e-bike** has to implement ratePerHour() only because the abstract base demands it – a sign that ratePerHour() is not a universal concept. A PricingRule abstraction would remove this.
- **DeliveryVan extends Car** guarantees "double the car rate" follows any change of the car rate, but a van is not a car in the real world, so van instanceof Car is true. Accepted for now; composition (a rate reference) would be cleaner.
- **Pricing is still distributed** (see B). It is acceptable at five rules; **if more than two or three further pricing rules arrive, move to a central PricingRule strategy** – the tests written here (35 + 31) are the safety net for that refactoring.
- **Pass validity** is checked on the entry date only: a visitor entering at 23:30 on the pass day and leaving the next day is still exempt. Documented limitation.
- **Cost of modification:** Vehicle and ParkingService were re-tested; the 35-test regression run did exactly that.

Final decision: MODIFY. Four of 14 classes changed (29 %), 10 stayed byte-identical, all original tests pass, all new tests pass, and the next vehicle type costs one class and one registration. The decision follows from the evidence: a redesign would have touched every class to remove a weakness that has

not yet cost anything, while retaining the design unchanged was impossible because a duration alone cannot express time-of-day pricing.

Conclusion

The parking system was analysed (Task 1), represented with justified types and a jagged array (Task 2), designed with explicit control logic and pseudocode (Task 3), and built from encapsulated, immutable-where-possible classes (Tasks 4–5) in an inheritance hierarchy with overriding, polymorphism and abstraction (Tasks 6–9), organised in packages (Task 10), and tested with 35 cases including a hand-calculated day (Task 11). The changed requirements of Task 12 were met by modifying 4 of 14 classes, leaving the original 35 tests passing and adding 31 new ones. Each alternative design was implemented or measured, and in every case the chosen design was retained for a stated reason, with its limits acknowledged.